高级计算机语言

知识体系整理与复习手册

Python 语言基础 · 机器学习入门 · Fortran 程序设计

课程定位	南京大学大气科学方向《高级计算机语言》课程
内容来源	课程 19 讲课件、教材与手写批注（fortran 重点标记）、讲义译文、全部作业
整理目标	覆盖每一讲的知识点、重难点、公式推导与图表，供反复翻阅复习

整理于 2026 年 10 月

阅读指南

本手册系统整理《高级计算机语言》课程的全部知识，按课程实际的三大模块组织：**第 1 部分 Python 语言基础**（对应课程 Python 入门讲义），**第 2 部分机器学习入门**（对应吴恩达机器学习讲义译文 6 讲及课程补充的深度学习应用），**第 3 部分 Fortran 程序设计**（对应第 1~19 讲课件与教材，篇幅最大、分量最重），附录汇总易错点、函数速查与参考卡。

使用方法建议：

- **备考复习**：先通读第 3 部分与附录 A 易错清单，再回头看讲义做对应习题；
- **查漏补缺**：目录条目可点击跳转；公式均为 Word 原生公式，可复制、可编辑；
- **代码练习**：文中所有代码均来自课件与作业原始素材，请动手在 gfortran / Python 中运行验证；
- 文中约定：**深蓝加粗**为定义与关键词，**红色加粗**为易错点与警告，等宽字体为代码与命令。

目录

第 1 部分　Python 语言基础

1.1 语言定位与特点

1.2 变量与作用域

1.3 运算符

1.4 注释

1.5 类型转换与用户输入

1.6 决策语句

1.7 循环

1.8 数值类型

1.9 字符串

1.10 四大容器：列表 / 元组 / 字典 / 集合

1.11 函数与递归

1.12 Lambda 匿名函数

1.13 类与对象

1.14 Python 与 Fortran 对照

第 2 部分　机器学习入门

2.1 定义与分类

2.2 单变量线性回归

2.3 多变量线性回归

2.4 逻辑回归

2.5 过拟合与正则化

2.6 偏差-方差诊断

2.7 支持向量机

2.8 神经网络

2.9 K-means 聚类

第 3 部分　Fortran 程序设计

3.1 语言发展史

3.2 编译环境与编译流程

3.3 程序基础

3.4 基本数据类型与声明

3.5 表达式与运算

3.6 派生类型 TYPE 与 KIND

3.7 输入输出与 FORMAT

3.8 流程控制

3.9 循环

3.10 数组

3.11 函数与子程序

3.12 全局变量、模块与接口

3.13 文件操作

3.14 指针

3.15 数据结构与算法

3.16 综合实战：双线性插值

3.17 编程规范与调试建议

3.18　面向对象编程：MODULE 的深入用法

3.19　编译器的高级使用与程序优化

3.20　课程各讲内容索引与作业清单

附录

附录 A　易错点与重点标注清单

附录 B　常用内在函数速查

附录 C　Fortran 90 参考卡精选

第 1 部分　Python 语言基础

本部分对应课程 Python 入门讲义（python 课件.pdf）。目标是：能用 Python 快速写脚本、处理数据，并为后续气象科研中调用 numpy、matplotlib、机器学习库等打下语法基础。学习重点是**变量的动态类型**、**缩进块结构**、**序列与字典的取舍**，以及与 Fortran 的**根本差异**。

1.1　语言定位与特点

Python 是 Guido van Rossum 开发、1991 年首次发布的高级编程语言，功能强大、**解释型**、**面向对象**，被称为“完美的脚本语言”。超过 75% 的数据科学岗位将其列为必备技能。

- **开源**：免费使用与分发；
- **高级语言**：不直接操作内存地址；
- **多编程范式**：面向对象、函数式、命令式均可；
- **解释型**：逐条解释执行，修改后无需重新编译；
- **动态类型**：变量无需事先声明类型；
- **跨平台**：Windows / Linux / macOS 通用；
- **代码量少**：同样功能所需代码行数显著少于 C/Java。

1.2　变量与作用域

变量是保留的内存位置。Python 中**不需要声明类型**，直接赋值即完成声明：

```
x = 5          # 无需 int x;
x = "hello"     # 同一变量可改绑不同类型
```

局部作用域：函数内定义的变量只能在该函数内使用，函数外无法访问。

```
def f():
    y = 10   # y 是局部变量
# print(y)      # 函数外访问会报 NameError
```

全局作用域：在函数（代码体）之外创建的变量，任何地方都可访问。要在函数内部**修改**全局变量的值，必须先用 `global` 关键字声明，否则赋值语句会在函数内**新建同名局部变量**，而不影响全局变量。

```
total = 0
def add(n):
    global total   # 声明要修改全局变量
    total = total + n
```

易错点：在函数里对全局变量直接赋值（不使用 `global`）不会报错，但修改的是局部副本，全局变量保持不变——这类逻辑错误极难察觉。

1.3　运算符

类别	运算符	说明与示例
算术	`+ - * / % //`	加、减、乘、除、取模、整除、幂；`2**3=8`，

类别	运算符	说明与示例
	**	7//2=3
赋值	= += -= *= /= 等	x += 1 等价于 x = x + 1
比较	== != > < >= <=	返回布尔值
逻辑	and or not	组合条件：a>0 and b>0
身份	is / is not	比较两个对象是否为**同一内存位置**的同一对象
成员	in / not in	检查元素是否属于元组、字符串、列表、字典等：'a' in 'abc'
位运算	& | ^ ~ << >>	按位与、或、异或、取反、左移、右移（逐位比较二进制数）

易错点：/ 在 Python 3 中总是返回 float（5/2=2.5）；要得到整数商必须用 //。身份运算符 is 比较的是**内存地址**，== 比较的是**值**，两者不可混用。

1.4 注释

```
# 单行注释
"""
多行注释：三引号字符串
也可作为多行字符串使用
"""
```

注释一般写在程序开头说明目的、作者与日期，也用于解释复杂语句。

1.5 类型转换与用户输入

函数	作用	示例
float(num)	把 int 转为 float	float(3) → 3.0
int(num)	把数值转为 int（截断小数）	int(3.9) → 3
complex(num)	把 int 转为复数	complex(3) → 3+0j
complex(a, b)	构造复数 a+bj（a 实部，b 虚部）	complex(1,2) → 1+2j

```
r = float(input("请输入半径："))
print("面积 =", 3.14159 * r * r)
```

input() 读取键盘输入，**返回值永远是字符串**，参与数值运算前必须先转换类型。

1.6 决策语句

Python 用 `if / elif / else` 做分支，**缩进（4 个空格）表示代码块**，没有大括号：

```
score = int(input("score:"))
if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 60:
    grade = 'C'
else:
    grade = 'D'
```

`break`：跳出**最近一层**循环，跳到循环后的下一条语句；`continue`：跳过本次循环剩余语句，直接进入下一次迭代。**注意**：两者只作用于循环，对 if 无效。

1.7 循环

while 循环：条件为真时反复执行，适合“执行次数事先未知”的情形。

```
i = 1
while i <= 5:
    print(i)
    i += 1    # 忘记这行会死循环
```

for 循环：适合“执行次数可以事先确定”的情形，常配合 `range()` 使用。

```
for i in range(1, 6):   # 1,2,3,4,5（左闭右开）
    print(i)
```

易错点：`range(a, b)` 不包含 b；while 循环忘记更新循环变量会导致死循环。

1.8 数值类型

类型	说明	示例
`int`	正负整数，无小数部分，**长度不受限制**（Python 3 只有 int，没有 long）	`1, 200, -10`
`float`	带小数点的浮点数；大数可用 E/e 缩写	`3.14, 1.5e10`
`complex`	实部 + 虚部，虚部用字母 **j** 表示	`3+4j, 10j, -30j`

1.9 字符串

字符串是字符序列，属于 `str` 类的对象。单引号与双引号等价；三引号用于多行字符串。

```
s = 'Hello'
s = "World"
s = '''第一行
```

```
第二行'''     # 三引号多行
s[0]        # 'H'，索引从 0 开始
s[1:3]      # 'el'，切片左闭右开
```

1.10 四大容器：列表 / 元组 / 字典 / 集合

这是 Python 数据组织的核心，也是与 Fortran 风格差异最大的地方。**列表**（`list`）是有序、**可变**的序列，用方括号；**元组**（`tuple`）是有序、**不可变**的序列，用圆括号（可省略）；**字典**（`dict`）是键值对集合，用大括号，**键唯一且不可变**、区分大小写；**集合**（`set`）是无序、不允许重复值的容器，元素可增删。

```
mytuple = (1, 25, 100)      # 元组：不可修改
list1 = [10, 25, 100]      # 列表：可修改
mystock = {'Product':'Earphone', 'Price':99, 'Quantity':3}
s1 = {1, 2, 3, 2}           # 集合：自动去重 -> {1,2,3}
```

维度	元组 tuple	列表 list
可变性	**不可变**，赋值后不能更改	**可变**，创建后仍可修改
定界符	圆括号（可省略）	方括号（强制）
速度	**较快**	较慢

1.11 函数与递归

函数以 `def` 开头，是“有组织的、可复用的代码块”，用于避免重复编写同样的任务。

```
def add(a, b):      # a, b 是形参
    return a + b
print(add(2, 3))    # 调用：函数名加括号
```

`return` 放在函数最后，用于**终止函数并返回结果**；带表达式时先求值再返回，不带表达式时返回 `None`。**递归**指函数调用自身，可以减少代码量：

```
def fact(n):
    if n <= 1:
        return 1
    return n * fact(n - 1)   # 递归计算 n!
```

1.12 Lambda 匿名函数

`lambda` 关键字定义**没有名字的函数**，可以有任意多个参数，但**只能有一个表达式**：

```
mul = lambda x, y: x * y     # 两个数相乘
sq  = lambda x: x * x        # 求平方
mx  = lambda a, b: a if a > b else b   # 两数较大者
```

适用于把“一次性使用的简单函数”直接内联到代码中（如排序的 key 函数）。

1.13 类与对象

Python 是面向对象语言：**类（class）是对象的模板（蓝图）**，用关键字 `class` 定义；**对象是类的实例**。对象具有三要素：**身份**（名称）、**状态**（属性，如自行车的颜色、型号、重量）、**行为**（方法，如骑车、刹车）。

```
class Bike:
    def __init__(self, color, model):
        self.color = color      # 状态（属性）
        self.model = model
    def ride(self):             # 行为（方法）
        print('riding')
b = Bike('red', 'M1')         # b 是 Bike 的一个实例
b.ride()
```

`__init__()` 类似 Java 的构造器：**创建对象时自动执行**，用来给对象属性赋初值。**注意**：类中方法的第一个参数固定是 `self`，代表对象本身。

1.14 Python 与 Fortran 对照（课程官方结论）

对比项	Python	Fortran
语言类型	**解释型**语言	**编译型**语言
代码长度	LOC 短	LOC 长
数组下标起点	**0 开始**	**1 开始**（默认）
计算速度	较慢	**快**
语法风格	清晰简洁，缩进即块	语法严格，格式固定
典型应用	Web 开发、数据科学、人工智能、机器学习	数值模拟、天气预报、高性能计算

要点：气象数值模式的核心计算用 Fortran 保证速度，数据后处理与机器学习用 Python 保证效率——两门语言各司其职，这也是本课程同时讲授两者的原因。

第 2 部分　机器学习入门

本部分对应吴恩达《机器学习》课程中文译稿（Lecture-01 ~ 06-译）以及课程补充的“深度学习与台风”应用案例。主线是：**监督学习的两大算法（线性回归、逻辑回归）→ 支持向量机 → 神经网络 → 无监督聚类（K-means）**，辅以一条贯穿始终的方法论主线——**过拟合与偏差-方差诊断**。

2.1　机器学习的定义与分类

Arthur Samuel（1959）的定义：机器学习是**让计算机在没有明确编程的情况下进行学习**的科学。Tom Mitchell（1998）给出“良定义”：

若程序在**任务 T** 上的表现（用**性能度量 P** 衡量）随着**经验 E** 的增加而改进，则称该程序**关于 T 和 P 从经验 E 中学习**。例：邮件程序观察用户标记垃圾邮件的行为（E），学习更好地过滤垃圾邮件（T），以分类正确率（P）衡量。

类型	输入数据	输出	典型问题
监督学习	带“正确答案”的样本 (x, y)	预测连续值（回归）或离散类别（分类）	房价预测（回归）、肿瘤良恶性判断（分类）
无监督学习	只有 x，没有标签	发现数据内部结构	聚类、市场细分、社会网络分析、天文数据分析
其他	—	—	强化学习、推荐系统

回归 vs 分类：回归预测**连续值**输出（如房价、未来三个月销量）；分类预测**离散值**输出（如 0/1、垃圾/非垃圾、账户是否被入侵）。

2.2　单变量线性回归

2.2.1　模型表示

记 m 为训练样本数，x 为输入特征，y 为目标变量。假设函数（hypothesis）把 x 映射到 y：

$$h_\theta(x) = \theta_0 + \theta_1 x$$

θ_0、θ_1 是模型参数，训练的目标是**选择参数使 hθ(x) 尽量接近 y**。

2.2.2　代价函数（损失函数）

$$J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

代价函数是**平方误差和的一半的平均**：系数 1/2 是为了求导后消去平方的 2，方便梯度下降。训练目标：

$$min_{\theta_0,\theta_1} J(\theta_0, \theta_1)$$

几何直觉：假设函数是**关于 x 的函数**（一条直线），代价函数是**关于参数 θ 的函数**（碗状曲面）。$\theta_0=0$ 时 $J(\theta_1)$ 是一条抛物线，最小值对应最佳直线。

2.2.3 梯度下降

思想：**从某个初始参数出发，沿“下山最快的方向”（负梯度）反复微调 θ**，直到收敛到局部（对线性回归是全局）最小值。更新规则（两个参数必须**同时更新**）：

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1), (j = 0,1)$$

对线性回归求偏导后得到：

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

要点	结论
学习率 α 太小	收敛**缓慢**
学习率 α 太大	可能越过最小值，**无法收敛甚至发散**
α 是否要随时间减小	**不需要**：越接近极小值导数越小，步长自动变小
局部最优	梯度下降可能停在局部最小值；线性回归的 J 是凸函数，只有一个全局最小值
同时更新	先算完所有新 θ，再统一赋值；**用更新过的 θ_0 去更新 θ_1 是错误**的
批量梯度下降	每一步都使用**全部 m 个训练样本**计算梯度

2.3 多变量线性回归

特征多于一个（面积、卧室数、楼层、房龄…）。记 n 为特征数，$x^{(i)}$ 为第 i 个样本，$x_j^{(i)}$ 为第 i 个样本的第 j 个特征。令 $x_0=1$，用向量记号：

$$h_\theta(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \cdots + \theta_n x_n$$

梯度下降每条规则形式与单变量完全一致，只是 x_j 扩展为 n+1 个特征（j=0 时 $x_0^{(i)}=1$）。**注意**：特征量纲相差过大（如面积 ~2000、卧室数 ~3）时，梯度下降收敛很慢，通常需要**特征缩放**（归一化到相近范围）。

2.4 逻辑回归（分类）

逻辑回归**解决分类问题**（y=0 或 1）。直接用线性回归 $h=\theta^T x$ 不合适——输出可能 >1 或 <0。解决方法是把线性输出压进 [0,1]，取 **Sigmoid（Logistic）函数**：

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

假设输出的含义：$h\theta(x)$ = 在输入 x、参数 θ 条件下 y=1 的**估计概率**。例：$h\theta(x)=0.7$ ⇒ 肿瘤有 **70% 的概率是恶性**。

决策边界：预测 y=1 当且仅当 $\theta^T x \geq 0$。$\theta^T x=0$ 就是决策边界——它把特征空间一分为二。添加多项式项可得到**非线性**决策边界（如圆）。

代价函数：若沿用平方误差，J(θ) 是**非凸**的（梯度下降易陷入局部最优）。逻辑回归使用对数损失（凸函数）：

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)}\log h_\theta(x^{(i)}) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))]$$

梯度下降的更新式与线性回归**形式完全相同**，区别仅在于 hθ(x) 现在是 sigmoid 函数。

多类分类（One-vs-all）：对每个类别 k 训练一个分类器 $h\theta^{(k)}$，预测"y 是否为 k 的概率"；对新样本 x，选择使 $h\theta^{(k)}(x)$ **最大**的类别。

2.5 过拟合与正则化

状态	表现	别名
欠拟合	模型太简单，训练集都拟合不好	高偏差（high bias）
正合适	训练集与测试集都表现良好	—
过拟合	特征太多/多项式阶数过高，训练集拟合极好但**无法泛化**到新数据	高方差（high variance）

解决过拟合的两条路：其一，**减少特征数量**（手动挑选或模型选择算法）；其二，**正则化**——保留全部特征，但**缩小参数 θ_j 的量级**，让假设更"简单"。

正则化线性回归：在代价函数中惩罚大参数（通常不惩罚 θ_0）：

$$J(\theta) = \frac{1}{2m}[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}\theta_j^2]$$

梯度下降更新式：θ_0 照旧；θ_j 先收缩再照常更新。**易错点**：λ 取**非常大**的值会让所有 $\theta_j \approx 0$、$h\theta(x) \approx \theta_0$ 成为水平线，导致**欠拟合**。

正则化逻辑回归：同上把 $\lambda/(2m)\cdot\Sigma\theta_j^2$ 加入对数损失（求和从 j=1 开始），梯度下降形式不变。

2.6 调试机器学习：偏差-方差诊断

模型在**新数据**上误差很大时，先做诊断再动手。**数据集划分**：训练集 60%、交叉验证集（validation）20%、测试集 20%（比例可调）。流程：

- 用训练集拟合各候选模型（如多项式次数 d=1,2,…,10）；
- 用**交叉验证集**误差挑选最优模型与超参数；
- 最后用**测试集**评估泛化误差（只在最后测一次）。

关键：若用测试集直接选模型，测试误差是泛化误差的**乐观估计**，因为模型选择这一"额外参数"本身是在测试集上拟合的。

诊断	训练误差 J_train	交叉验证误差 J_cv	对策
高偏差（欠拟合）	高	高，且 J_cv ≈ J_train	**增加特征、加多项式项、减小 λ**；加数据无用

诊断	训练误差 J_train	交叉验证误差 J_cv	对策
高方差（过拟合）	低	J_cv ≫ J_train（很高）	**减少特征、增大 λ、增加训练数据**

学习曲线：误差随训练集大小的变化曲线。高偏差时 J_train 与 J_cv 都较高且趋向平齐，更多数据**无济于事**；高方差时两条曲线之间有**大间隙**，更多数据**很可能有帮助**。

神经网络与过拟合：小网络参数少、计算便宜、易欠拟合；大网络参数多、计算昂贵、易过拟合——大网络过拟合可用正则化解决，因此"更大的网络 + 正则化"通常优于小网络。

2.7 支持向量机（SVM）

SVM 是逻辑回归的"另一种视角"。逻辑回归希望在 y=1 时 $\theta^T x \gg 0$（远大于 0，而非仅仅 >0）；SVM 把这个要求进一步**收紧为 $\theta^T x \geq 1$**，并把代价函数换成**铰链损失（hinge loss）**形式。这使它成为**大间隔分类器（large margin classifier）**：决策边界不仅把两类分开，而且**离最近的样本（支持向量）尽可能远**。

$$min_\theta C \sum_{i=1}^{m} [y^{(i)} \text{cost}_1(\theta^T x^{(i)}) + (1 - y^{(i)}) \text{cost}_0(\theta^T x^{(i)})] + \frac{1}{2} \sum_{j=1}^{n} \theta_j^2$$

核函数（kernel）：把样本映射到新特征 f_j，使非线性问题线性可分。常用**高斯核（RBF）**：

$$f_j = \exp(-\frac{\|x - l^{(j)}\|^2}{2\sigma^2})$$

其中 $l^{(j)}$ 是**地标（landmark）**——直接取每个训练样本作为地标；x 离 $l^{(j)}$ 近则 $f_j \approx 1$，远则 $f_j \approx 0$。**注意**：并非任意相似度函数都是有效核函数，须满足 **Mercer 定理**的技术条件（否则优化会发散）；其他可选核有**多项式核、字符串核、卡方核、直方图交集核**等。

参数	取值效果
C 大	低偏差、**高方差**（更易过拟合，间隔小）；C 小则相反
σ^2 大	特征 f 变化**平滑**：高偏差、低方差；σ^2 小则 f 变化陡峭：低偏差、高方差

逻辑回归 vs SVM 的选型：n 远大于 m（特征多、样本少）→ 逻辑回归或**线性核** SVM；n 小、m 中等 → **高斯核 SVM**；n 小、m 很大 → 手工构造更多特征后用逻辑回归或线性核 SVM。神经网络多数场景可用，但训练较慢。

2.8 神经网络

2.8.1 动机与结构

动机：某些问题（如 50×50 像素 = 2500 个像素输入，二次特征约 **300 万**个）二次项特征爆炸，必须学习自己的特征。网络由**输入层、隐藏层、输出层**组成：第 j 层第 i 个单元的激活值记 $a_i^{(j)}$，g 为 sigmoid 激活函数，$\Theta^{(j)}$ 为从第 j 层映射到第 j+1 层的权重矩阵（若第 j 层 s_j 个单元、第 j+1 层 s_{j+1} 个单元，则 $\Theta^{(j)}$ 为 $s_{j+1} \times (s_j+1)$ 维）。

前向传播（向量化）：

$$a^{(j+1)} = g(\Theta^{(j)}a^{(j)})$$

逐层计算，最终 $h\Theta(x)=a^{(L)}$。每层在**上一层的激活值基础上组合出更复杂的特征**——网络自己学习特征。

2.8.2 用逻辑单元构造逻辑运算（直觉）

一个 sigmoid 单元近似一个逻辑门：取权重（−30, 20, 20）并激活 $g(-30+20x_1+20x_2)$，可得到 **AND 门**；取（−10, 20, 20）得到 **OR 门**；单个负权重得到 **NOT 门**。把 AND、OR、NOT 逐层**堆叠**，即可构造 **XOR/XNOR** 等非线性分类器——这就是“多层网络表达能力”的直观来源。

2.8.3 代价函数与反向传播

多类分类（K 类，y 为 K 维向量）的代价函数是逻辑回归代价的逐类、逐样本推广：

$$J(\Theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}[y_k^{(i)}\log(h_\Theta(x^{(i)}))_k + (1-y_k^{(i)})\log(1-(h_\Theta(x^{(i)}))_k)]$$
$$+\frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i}\sum_{j}(\Theta_{ij}^{(l)})^2$$

反向传播用于高效计算 J 关于每个权重的偏导。核心是定义**误差项** $\delta_j^{(l)}$（第 l 层节点 j 的“误差”）：对输出层 $\delta^{(L)} = a^{(L)} - y$，再逐层反推：

$$\delta^{(l)} = (\Theta^{(l)})^T\delta^{(l+1)}g'(z^{(l)})$$

对每个训练样本做一次前向传播（得各层 a）和一次反向传播（得各层 δ），按 $\Delta^{(l)} \mathrel{+}= \delta^{(l+1)}(a^{(l)})^T$ 累加；最后 $D=\Delta/m$ 并加入正则项 $\lambda/m\cdot\Theta$，即得到 $\partial J/\partial\Theta$。

- **随机初始化（打破对称性）**：Θ 全零初始化会让每个隐藏单元学到完全相同的函数；应在小范围内随机取值（如 [−ε, ε]）。
- **梯度检查**：用数值差分 $(J(\theta+\varepsilon)-J(\theta-\varepsilon))/(2\varepsilon)$ 与反向传播梯度比对，确认无误后**关掉梯度检查**再训练（它极慢）。
- **训练流程**：随机初始化 → 前向传播 → 计算 J → 反向传播求偏导 → 梯度检查 → 用梯度下降/高级优化器最小化 J。
- **结构选择**：输入单元数=特征维数；输出单元数=类别数；隐藏层合理默认 1 层，或每层单元数相同；通常**单元越多越好**。

2.8.4 训练神经网络的实践观察（TensorFlow Playground）

- 第一隐藏层学到简单模式，第二层把简单模式**组合**成复杂模式——层数越多模式越复杂；
- 激活函数用 ReLU 替代 tanh 通常**收敛更快**，但边界可能偏线性（与 ReLU 形状有关）；
- 小网络（参数少）多次训练结果不稳、易**欠拟合**；足够大的网络几乎**不会陷入糟糕的局部极小**；
- 深网络训练变慢、容易长期停在平台上，高层神经元比低层进化快——**梯度消失**问题，可用更好的初始化、优化器（AdaGrad/Adam）或批归一化缓解。

2.8.5 应用案例：DeepTCNet 台风强度与尺度估计

课程结合气象背景介绍了**物理增强的深度学习**模型 DeepTCNet：基于**卷积神经网络（CNN）**，利用葵花 8 号（Himawari-8）与 GOES-16/17 卫星红外云图，实时估计热带气旋的强度与尺度——输出**最大风速 Vmax 与中心气压 Pc**、**最大风半径 RMW**，以及 **R34/R50/R64 四象限风速半径**。该工作发表于 Mon. Wea. Rev.（Zhuo & Tan, 2021）。演示平台网址：https://forecast.nju.edu.cn/deeptcnet。

2.9 K-means 聚类（无监督）

输入只有样本 $x^{(1)},\ldots,x^{(m)}$（无标签），目标是把数据自动分成 K 个**簇（cluster）**。算法（循环两步）：

- **簇分配**：对每个样本 i，把它赋给**离它最近**的簇中心：$c^{(i)}$ = 距离 μ_k 最小的 k；
- **移动簇中心**：每个簇中心 μ_k 更新为**分配给它的所有点的均值**。

$$c^{(i)} = \arg min_k \|x^{(i)} - \mu_k\|^2, \mu_k = \frac{1}{|C_k|}\sum_{x^{(i)} \in C_k} x^{(i)}$$

优化目标（失真函数）：K-means 最小化每个样本到其簇中心距离平方之和。

$$J = \frac{1}{m}\sum_{i=1}^{m} \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

- **随机初始化**：随机选 K 个**训练样本**作为初始中心；不同初始化会得到不同结果（局部最优），通常**多跑 50~100 次**，取失真函数 J 最小的一次；
- **K 的选择**：看“肘部法则”——J 随 K 的下降曲线在拐点处取 K；若曲线平滑无拐点，则**以下游任务的表现**决定 K（如 T 恤尺码 S/M/L 对应 K=3）；
- **非分离簇**同样有效：K-means 对相互交叠的簇也适用。

第 3 部分　Fortran 程序设计

本部分对应课程第 1~19 讲及教材《Fortran 95/2003 程序设计》。Fortran = **FORmula TRANslator（公式翻译器）**，是世界上最早的高级程序设计语言，至今仍是**数值、科学、工程计算**（尤其是气象、天气预报、高性能计算）的核心语言。学习主线：语言要素 → 流程控制 → 数组 → 过程 → 文件 → 数据结构与算法 → 气象插值实战。

3.1　Fortran 语言发展史

年份	事件
1951	IBM 的 John Backus 针对汇编语言的缺点着手研发 Fortran
1954	Backus 发布第一个版本 FORTRAN I
1957	第一个 Fortran 编译器在 **IBM 704** 计算机上实现，首次成功运行 Fortran 程序
1958	推出 FORTRAN II，引入**子函数**等概念
1962	推出 FORTRAN IV；此前 FORTRAN III 因严重缺陷未能在计算机上实现
1966	ANSI 公布美国国家标准 **FORTRAN 66**
1972	ISO 在 FORTRAN 66 基础上制定三级国际标准（基本级、中间级、完全级）
1978 / 1980	ANSI 公布 **FORTRAN 77**；1980 年成为 ISO 1539-1980 国际标准，向下兼容 66，具有结构化特征、扩充字符处理
1992	ISO 公布 **FORTRAN 90**：加入面向对象概念、提供指针、加强数组/矩阵运算、改良版面格式
1997	ISO 公布 **FORTRAN 95**：加强对并行计算的支持
2003	最新标准 FORTRAN 2003（课程采用的教科书基线）

语言实现方式：Fortran 是**编译式**语言——源代码先由编译器翻译成目标文件，再经链接器与库链接生成可执行程序；而 MATLAB、BASIC 属于**直译式（解释式）**语言，直接在 CPU 中运行。**要点**：同一个程序代码，不同编译器生成的执行文件并不相同。

3.2　编译环境与编译流程

3.2.1　编译 → 链接 → 执行

FORTRAN 源程序经**编译（compile）**得到目标文件，再与**库（Library）链接（link）**生成**可执行程序（executable）**。编译器评价标准：翻译是否正确、执行文件运行效率、执行码长短、编译耗时、调试工具完整程度、帮助是否清楚、扩充功能。

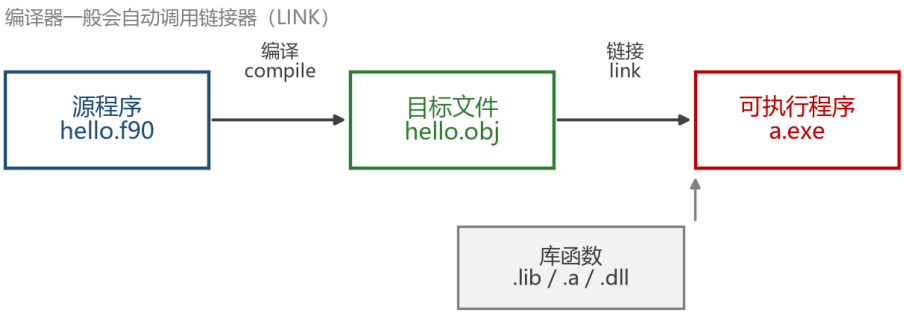


图 3.2-1　从源程序到可执行程序：编译产生目标文件，链接把目标文件与库函数合成可执行程序（编译器通常自动调用 LINK）

3.2.2　常用编译器

- DOS 时代：MS Fortran、NDP Fortran；Windows：Microsoft Fortran PowerStation 1.0~4.0 → Digital Visual Fortran 5.0 → **Compaq Visual Fortran（CVF）6.0~6.6**；
- Linux/Windows：**G77、G95、GFortran**（gfortran 是 GNU 免费编译器，课程主用）；
- 商业：Portland Group Fortran（PGI）、Intel Fortran（ifort）、Lahey Fortran、SGI/Sun/HP Fortran（Unix）；
- **注意**：CVF 已停止开发十多年，支持 Fortran 90、部分支持 95；其安装路径**不能出现中文字符**。Silverfrost Fortran 的 kind 值与标准不兼容，属非标准扩展。

3.2.3　Cygwin 与 Linux 命令

Cygwin 是运行在 Windows 上的类 UNIX 环境：它提供一套在 Win32 系统下实现 **POSIX 系统调用 API** 的库，以及 GNU 开发工具集（GCC、GDB）与常用 UNIX 程序，用于把 Linux 软件移植到 Windows。气象工作者需要熟悉 Linux，因为数值模式都跑在 Linux 和超级计算机上。

```
# 编译 hello.f90（g77/g95/ifort 同理）
gfortran hello.f90      # 无错误则生成 a.out（相当于 Windows 的 .exe）
./a.out                  # ./ 表示当前目录
pgf90 -o hello.exe hello.f90   # 自定义可执行文件名
```

命令	功能	命令	功能
cd / cd ..	进入目录 / 返回上一级	tar	解压文件夹
ls	显示文件	echo	显示字符
vi	编辑文件	less / more	分页显示文件
rm	删除文件	export	设置环境变量
mkdir	新建文件夹	chmod	设置文件执行权限
cp / mv	拷贝 / 移动文件	mount / umount	加载 / 卸载设备
grep	查找	diff	比较文件及列表差异
man	查看命令手册	cat	显示文件内容
pwd	当前文件夹路径	:wq!	vi 中写入保存并退出（w

命令	功能	命令	功能
			写、q 退出）

gfortran 调试小流程：`vi test.f90` 后按 `i` 进入插入模式（底部出现 --INSERT--），按 Esc 退出编辑模式，输入 `:wq` 保存退出；再 `gfortran test.f90`（若提示 command not found 说明安装时漏装包），`./a.exe` 运行。

3.3 Fortran 程序基础

3.3.1 字符集与名称

Fortran 字符集：**26 个英文字母（大小写不分**，INTEGER 与 iNteger 等价）、**数字 0-9**、以及约 22 个特殊符号 `: = + - * / ( ) , . ' " ! % & ; < > ? $ _`（含空白字符）。

名称规则（变量、函数、程序单元）：最长 **31 个字符**，由英文字母、数字、下划线（或 $）组成，**第一个字符必须是英文字母**。**易错**：`_Wrong`（下划线开头）、`A.1`（含点）、`5Q`（数字开头）都不合法；`Number`、`U_wind`、`name234` 合法。**关键字**（IF、READ、INTEGER…）不得用作名字。

3.3.2 程序单元结构

一个可执行程序由**一个主程序（program）**和任意数目的**外部子程序（subroutine/function）**、**模块（module）**、**块数据单元（block data）**组成。主程序与子程序的基本形式相同，都由三部分构成：

```
PROGRAM my_first_program
  INTEGER :: I, J, K          ! 声明部分：说明语句
  READ(*,*) I, J              ! 执行部分：执行语句
  K = I * J
  WRITE(*,*) 'RESULT = ', K
  STOP                        ! 终止部分
END PROGRAM my_first_program
```

- **说明语句**：类型关键字 `::` 变量名 `1, 变量名 2 …`（如 `INTEGER :: A, B, C=5`）声明变量类型；
- **执行语句**：关键字 变量名…（如 `READ *, X, Y`、`PRINT *, X+Y`）执行具体操作；
- **终止语句**：STOP 结束执行、END PROGRAM 告知编译器“没有更多语句要编译了”。

3.3.3 书写格式：固定格式与自由格式

固定格式（Fixed Format）是 F77 及以前的旧写法（*.F / *.For），为配合早期**穿孔卡片**输入而设计，版面限制严格；**自由格式（Free Format）**是 F90 之后的新写法（*.F90），取消版面限制。**要求**：建议**用自由格式编写**程序，但要**能读懂固定格式**的经典气象程序。

列位置	固定格式规则
第 1 列	字母 C、c 或星号 * ⇒ 整行为注释，不编译
第 1~5 列	数字 ⇒ 行号（标号）；否则只能空格
第 6 列	非 0 字符 ⇒ 本行是上一行的**续行**
第 7~72 列	程序代码区

列位置	固定格式规则
第 73 列以后	不使用，会被忽略（有的编译器报错；部分可扩展到 132 列）

自由格式注意：！后的文本都是注释；每行最多 132 字符；行号放在每行最前面；**续行符是 &**，放在上一行末尾或下一行开头：

```
write(*,*) &        ! 行尾 & 表示连接下一行
     "Hello"
wr&
&ite(*,*) "Hello"    ! & 也可拆开关键字
```

编程建议：**尽量多写英文注释**（可移植）；语法尽量规范；命令之间的空格无意义，可借缩进让程序直观易读。

3.4 基本数据类型与声明

3.4.1 五种基本类型

数据类型告诉编译器“用何种方式在内存中存储数据”。整数以二进制直接存储，浮点数以二进制指数（规格化）方式存储，因此**不同类型数据要转换后才能互通**。

类型	占用与表示	取值范围 / 精度
`INTEGER` 长整型	32 bits（4 bytes）	$-2^{31} \sim 2^{31}-1$（约 $\pm 2.1\times10^{9}$）
`INTEGER` 短整型	16 bits（2 bytes）	$-2^{15} \sim 2^{15}-1$（约 ± 32768）
`REAL` 单精度	32 bits，有效 6~7 位	最大 $\pm 3.4\times10^{38}$，最小 $\pm 1.18\times10^{-38}$
`REAL` 双精度	64 bits，有效 15~16 位	最大 $\pm 1.79\times10^{308}$，最小 $\pm 2.23\times10^{-308}$
`COMPLEX` 复数	a+bi，a、b 各为浮点数	分单精度 / 双精度复数
`CHARACTER` 字符	每个字符 1 byte	可记录键盘输入的任何内容（数字、文本、符号）
`LOGICAL` 逻辑	TRUE / FALSE	二进制中 1 代表真、0 代表假

3.4.2 IMPLICIT 与显式声明

Fortran 传统上**变量不声明也能使用**：以 **I、J、K、L、M、N 开头的变量默认为整型，其余默认为实型**。**易错**：这种隐式类型极易造成人为错误（如变量名手误成另一个默认类型）。**强烈建议每个程序单元都在开头写 `IMPLICIT NONE`**，强制所有变量先声明后使用（类似 C）。`IMPLICIT` 语句必须紧接在 `PROGRAM`（或 function/subroutine）语句之后。

```
implicit integer(A,B,C)     ! A,B,C 开头的变量都视为整数
implicit integer(A-F,I,K)    ! A-F 与 I,K 开头为整数
```

```
implicit real(M-P)            ! M-P 开头为实数
implicit none                 ! 关闭默认类型，全部变量必须声明（推荐）
```

3.4.3 PARAMETER、DATA 与变量初值

```
! F90 写法
real, parameter :: pi = 3.14159   ! 命名常量，执行期不可改变
integer :: a = 1                    ! 声明同时赋初值
! F77 写法
real pi
parameter(pi=3.14159)
integer a
real b
data a, b /1, 2.0/                  ! DATA 按顺序赋初值
```

PARAMETER 常量在程序中**不能改变**，若被修改编译会报错；常量在编译时即以二进制写入寄存器，可**提高执行速度**。DATA 是 F77 的赋初值方式（按顺序），F90 可直接在声明中赋初值（**此时中间的 :: 不能省略**）。**易错**：DATA 语句是声明的一部分，只能放在执行语句之前。

3.4.4 EQUIVALENCE 等价声明

EQUIVALENCE 把两个以上的变量声明为**使用同一个内存地址**：改变其中一个，另一个随之改变。优点：节省内存（临时变量共用一块内存）、精简代码（用变量代替频繁使用的数组片段）。

```
INTEGER A, B
EQUIVALENCE (A, B)      ! A、B 使用同一块内存
! equivalence(array(1,1,5), a)   ! 数组片段与变量等价
```

3.4.5 声明的位置

声明必须位于**可执行语句之前**：一旦出现数值计算和输入输出命令，就不能再声明变量。声明在编译后**不产生机器码**，它只是请编译器预留内存空间。**易错**：把 `real b` 写在 `read(*,*) a` 之后 → 编译错误。

3.4.6 复数类型（COMPLEX）

复数由实部与虚部两个浮点数组成（a+bi），分为**单精度复数**（占 8 bytes）与**双精度复数**（占 16 bytes）。赋值时把实部、虚部用括号或 CMPLX 函数组合起来，`(1.0, 2.0)` 就是 1+2i：

```
complex :: c          ! 单精度复数（默认 kind=4）
complex(8) :: c8      ! 双精度复数（实部、虚部都是双精度）
c  = (1.0, 2.0)       ! 1 + 2i
c  = cmplx(1.0, 2.0)  ! 与上式等价（虚部省略时默认为 0）
c8 = (1.0_8, 2.0_8)
```

操作	写法	说明
取实部 / 虚部	`real(c)` / `aimag(c)`	`aimag` 是复数专用内在函数，取虚部
合成复数	`cmplx(x, y [,kind])`	把两个实数合成复数

操作	写法	说明
共轭复数	`conjg(c)`	a+bi → a−bi
模（绝对值）	`abs(c)`	√(a²+b²)
运算	`+ - * /  **`	四则运算按复数规则进行，如 (1+2i)*(3+4i) = −5+10i

易错：变量声明成实型却接收 `cmplx` 的结果，只会得到实部；复数也**不能**用于 `select case` 的选择表达式。

3.5 表达式与运算

3.5.1 算术表达式与优先级

优先级从高到低：**括号 ()** → **乘幂 **** → **乘除 * /** → **加减 + -**；同级从左向右；括号越内层越优先。

- 乘幂要**连用两个星号**：2^2 必须写成 `2**2`，不能写成上标；
- **乘号不能省略**：(A+B)(C+D) 必须写成 `(A+B)*(C+D)`，`2*A` 不能写成 `2A`；
- 除法只能写成分数线的水平形式，即 `a/b`。

3.5.2 整数除法与混合运算

极重要：**整数除以整数得整数（截断）**。`5/2 = 2`（不是 2.5）；`16**(1/2)` 先算括号里 `1/2 = 0`，于是等于 `16**0 = 1`（不是 4！）。

```
integer :: a=1, b=2
real :: c
c = a/b            ! c = 0：先按整数算 1/2=0，再赋给实型
c = real(a)/real(b)  ! c = 0.5：先转换为实型再除
c = (a*1.0)/(b*1.0)  ! 也可用 *1.0 转换；*1.0_8 可提精度
```

混合运算指算式中的数字类型不完全相同（如整数+实数）。编译器有时会自动转换，但**不保证符合要求**，最好由程序员显式转换。浮点转整数用 `int(r)`（截断）。

3.5.3 关系运算、逻辑运算与字符运算

F90 符号	F77 写法	含义
==	.EQ.	等于
/=	.NE.	不相等
>	.GT.	大于
>=	.GE.	大于等于
<	.LT.	小于
<=	.LE.	小于等于

逻辑运算符	含义
.AND.	交集：两边都成立，整个条件才成立
.OR.	并集：两边只要有一个成立，整个条件就成立
.NOT.	逻辑反向：后面的式子不成立，整个式子才成立
.EQV.	两边逻辑结果**相同**时成立
.NEQV.	两边逻辑结果**不同**时成立

逻辑表达式可以自由组合，可用括号确定运算顺序；**比较运算优先级高于逻辑运算**（`if (a>=80 .and. a<90)` 等价于 `if ((a>=80) .and. (a<90))`）。字符操作符是 `//`（连接字符串）。逻辑变量可直接接收比较结果：`logical_var = A > B`。

易错：`a>0 .and. b>0 .or. a<0 .and. b<0` 表示“同为正或同为负”；写成 `.EQV.` 版本留给读者思考（提示：两数符号相同 ⇔ 比较结果的逻辑值相等）。

3.6 派生类型 TYPE 与 KIND

3.6.1 自定义数据类型 TYPE

记录一个气象站的多要素（站名、日期、经纬度、高度、风温湿压）需要很多变量；用 TYPE 可以把它们打包成一个**站点类型**，之后像使用内置类型一样使用它：

```
type :: station                    ! 建立 station 类型
  character(len=10) :: sta_name
  integer(kind=2)   :: iyear, imon, iday
  real(kind=4)      :: sta_lat, sta_lon, sta_ht
  real(kind=4)      :: u, v, t, td, rh, prs
end type station
type(station) :: a                    ! 声明变量
a = station("Nanjing",2005,4,23,32.,118.48,89.,5.,1.4,27.,24.,75.,1001.2)
read(*,*) a%sta_name                 ! 用 % 访问元素（也可用 .）
```

3.6.2 KIND 种类参数

KIND 声明整数、浮点数占用的内存字节，根据**有效位数与值域范围**选择，可增强程序跨平台能力。两个询问函数：

- `SELECTED_INT_KIND(n)`：返回能记录 **n 位整数**的 kind 值；返回 −1 表示无法提供所需值域；
- `SELECTED_REAL_KIND(n,e)`：返回能记录 **n 位有效数字、指数到 e 位**的浮点数 kind 值；返回 −1 表示有效位数不满足、−2 表示指数范围不满足。

```
integer, parameter :: long_int = selected_int_kind(9)
integer(kind=long_int) :: a = 12345678
real :: c = 1.23456789D45   ! D 指数表示双精度常数
```

数值常数也可直接带 kind：`a=1.0_4`、`b=2.0_8`（数字后加下划线和 kind 值）。赋值时 kind 不常用，但**计算中临时提高精度**时很有用。

3.7 输入输出与 FORMAT

3.7.1 基本输出与输入

```
write(*,*) "hello"     ! 第一个 * 是输出位置（屏幕），第二个 * 是不特别设置格式
print *, "hello"       ! print 是 write 的简化
write(6,*) "hello"     ! 屏幕输出的 unit 编号是 6，unit=* 即 unit=6
write(unit=6, fmt=*) "Hello"
read(*,*) A, B          ! 从键盘读入
read(5,*) A, B          ! 键盘输入的 unit 是 5
```

易错：每次执行 write 后**自动换行**（与 C 不同）；输出字符串里的双引号要**连写两个双引号**转义（"He said ""hi"""）。

3.7.2 FORMAT 格式控制

```
a = 12.3456
write(*,*) a        ! 输出 12.3456（默认格式）
write(*,100) a      ! 使用行号 100 的格式
100 format(f5.2)    ! 输出 12.34：5 个字符宽、2 位小数、2 位整数、小数点占 1 位
```

FORMAT 的目的：**有计划地设计输出版面**。简写形式：write(*,"(1x,f5.2)") a。优点：减少行数与行号、易读；缺点：格式复杂时代码混乱、相同格式不能复用。格式串里的逗号可省略成 (1xf5.2)；重复的相同格式写成 (2a3)；重复的不同格式写成 (3(1xf5.2))；字符串可直接写入格式：

```
write(*,"(a4,I1)") "1+2=", 1+2     ! 输出 1+2=3
write(*,"('1+2=',I1)") 1+2            ! 等价写法（F77 单引号需加倍）
```

格式可以放在**字符串变量**中，运行期动态改变：

```
character(len=10) :: fmtstring
fmtstring = "(f5.1)"
write(*,fmtstring) 123.4
fmtstring(3:3) = "8"      ! 变成 (f8.1)，容纳更大的数
```

3.7.3 格式控制符速查

控制符	含义	示例与输出
Aw	w 个字符宽度输出字符串	(A10) "hello" → □□□□□hello
Dw.d	指数型浮点（w≥d+6），D 指数	(D15.7) 123.45 → □□0.1234500D+03
Ew.d	指数型浮点（w≥d+6），E 指数	(E15.7) 123.45 → □□0.1234500E+03
Fw.d	w 个字符宽浮点数、d 位小数	(F8.4) 123.45 → 123.4500
Iw	w 个字符宽整数	(I5) 100 → □□100
Lw	w 个字符宽逻辑值 T/F	(L5) .TRUE. → □□□□T

控制符	含义	示例与输出
nX	跳过 n 个位置	(5X,I3) 100 → □□□□□100
/	换行输出	(I3//I3) 10,10 → 10、空行、10

常见错误：变量类型与格式类型不符（整数却用 A 格式）；**字段宽度不够时输出 ***（如 (I3) 输出 10000）；早期行式打印机把每行第一个字符当控制字符，故传统上格式开头加 1X。

3.8 流程控制

3.8.1 IF 语句族

```
if ( speed > 100.0 ) then      ! 逻辑判断式
  write(*,*) "Slow down."      ! 成立时执行
end if
! 单行形式：if (speed > 100.0) write(*,*) "Slow down"

if ( precip > 50.0 ) then
  print *, "Heavy Rain"        ! 成立执行
else
  print *, "Precip = ", precip    ! 不成立执行
end if
```

IF-ELSE IF 多重判断：一次列出多个条件与多个程序块，**最多只有一个条件成立**、最多只有一块被执行；最后的 else 块可省略，若全部不成立则什么都不做，程序从 end if 之后继续。

```
if (score>=90 .and. score<=100) then
  grade='A'
else if (score>=80 .and. score<90) then
  grade='B'
else
  grade='?'       ! 处理 score<0 或 >100 的意外输入，推荐保留 else
end if
```

三种写法的取舍：**ELSE IF** 找到第一个成立的即停止（效率高）；**独立多个 if** 每个表达式都执行（效率最低）；**嵌套 if** 层层展开，逻辑最灵活。

3.8.2 浮点数不能直接判断“等于”

高频考点：浮点计算受**有效位数限制**必然产生舍入误差，理想中的等号不一定会成立。例如 a=SQRT(b)**2-b（b=3.0）理论上是 0，实际是接近 3 但未必等于 3 的数，于是 a==0.0 的逻辑值是 .FALSE.——而且**不同编译器的结果也会不同**。正确做法是用**绝对或相对误差**判断：

```
if ( abs(b-a) < 0.000001 ) then ...         ! 绝对误差
if ( abs((b-a)/b) < 0.000001 ) then ...   ! 相对误差
! abs 是取绝对值的内在函数
```

3.8.3 SELECT CASE

```
select case( score )
case( 90:100 )   ! 90~100（含端点）
  grade='A'
case( 80:89 )
  grade='B'
case( 0:59, 101: )   ! 多个区间并列
  grade='?'
case default      ! 都不匹配时执行（可省略）
  grade='E'
end select
```

约束：SELECT CASE 只能判别**整数、字符、逻辑**类型（不能用浮点数和复数）；`case()` 中的数值必须是**常数**，不能用变量（用 `parameter` 声明的常量可以）；`case(1:)` 表示 ≥1，`case(:5)` 表示 ≤5，`case(1:5)` 表示 1~5。

3.8.4 GOTO、算术 IF 与 PAUSE / CONTINUE / STOP

- `GOTO 行号`：跳到指定行号处继续执行，是 F77 遗留，**不建议使用**（破坏结构化、使程序混乱）；计算型 `GOTO(10,20,30) I/N` 按变量值分别跳转；
- **算术 IF**：`IF(C) 10,20,30` 表示 C<0 跳 10、C=0 跳 20、C>0 跳 30；
- `PAUSE`：暂停并等待按 ENTER 继续（大量屏幕输出时用），一些编译器要求输入 go 后回车；
- `CONTINUE`：空操作，表示“继续向下执行”，F77 中配合 GOTO 作循环终点；
- `STOP`：结束程序执行，如 `IF() STOP` 在读到不合理数值时输出警告并中止。

3.8.5 字符的比较

字符保存的是 ASCII 码，比较即比码值：`'a'<'b'`（97<98）、`'A'<'a'`（65<97）；字符串按字母顺序**逐个字符**比较：`'abcd' < 'axyz'`。用内在函数 `ichar` 取得字符编码。

3.9 循环

3.9.1 DO 计数循环

```
do counter = 1, 10, 1     ! 计数器 = 初始值, 终止值, 增量（增量缺省为 1）
  write(*,*) "Happy Birthday", counter
end do                     ! 或 enddo
! 例：10! 累乘
do i = 1, 10
  ans = ans * i
end do
```

F77 写法：`DO 100 I = 1, 10 … 100 CONTINUE`（行号 100 为循环终点）。**循环规则**：

- 初始值、终止值、增量在**进入循环时**就被读入一次，循环中改变这些变量的值**不影响循环执行**；
- 增量可正可负（如 `do i=10,1,-1`），缺省为 1；
- **易错**：**循环体内不允许修改计数器**（`i = i + 1` 会编译错误）；正常结束后计数器一定大于终止值。

3.9.2 DO WHILE 循环

```
do while( counter <= limit )   ! 条件成立时反复执行
  ans = ans + counter
  counter = counter + 2
end do
```

DO 循环用于**可预知执行次数**的循环；DO WHILE 用于**不能预知次数**的循环（如猜数字直到猜对）。**注意**：DO WHILE 在**进入循环前**就检查条件，条件不成立则一次也不执行；要保证至少执行一次，可用 `do while(.true.)` + `exit` 组合。

3.9.3 CYCLE 与 EXIT

```
do floor = 1, dest
  if ( floor == 4 ) cycle   ! 跳过本次循环剩余语句，回到循环开头
  write(*,*) floor
end do

do i = 1, 1000
  sum = sum + a(i)
  if ( a(i) == 0 ) exit     ! 提前跳出循环（对 DO 和 DO WHILE 都有效）
end do
```

3.9.4 署名循环与嵌套

```
loop1: do i = 1, 3          ! 外层循环取名 loop1
  loop2: do j = 1, 3        ! 内层循环取名 loop2
    if ( i == 3 ) exit loop1    ! 直接跳出外层循环
    if ( j == 2 ) cycle loop2   ! 重做内层循环
    write(*,"('(',I2,',',I2,' )')") i, j
  end do loop2
end do loop1
```

多层循环嵌套尽量**先执行内层、后执行外层**（配合数组的列主存储效率最高）。署名让多层循环里 `exit` / `cycle` 的目标一目了然，减少出错。

3.9.5 循环典型应用

- 等差数列求和 1+2+…+100：`do counter=1,100; ans=ans+counter; end do`；
- 斐波那契数列前 10 个：用 `fn_1`（前一项）、`fn_2`（前两项）滚动更新 `fn = fn_2 + fn_1`；
- 凯撒加密：`string(i:i) = char(ichar(string(i:i)) + key)`，解密把 +key 换成 −key（`len_trim` 取字符串实际长度）。

3.10 数组

3.10.1 概念：秩、维界、长度、大小、形状

- 数组是**类型相同**的一组标量数据的**有序集合**，是科学计算中向量和矩阵的反映；
- **秩（rank）** = 维数，F90 规定最多 **7 维**；

- **维界**：每一维由下界和上界指定（下界:上界），可正可负可为零，但上界 > 下界；**维长（extent）** = 上界 − 下界 + 1；
- **大小（size）** = 各维长度的乘积（可为 0）；**形状（shape）**由秩与各维长度决定；
- 例：`REAL A(10,3,2)` 的秩为 3、大小为 60、形状为 (10,3,2)；`SHAPE(A)` 返回一维数组 (10,3,2)。

3.10.2 声明与下标

```
integer a(10)                      ! 最简单
integer, dimension(10) :: a        ! 推荐写法
integer a                         ! F77：先声明类型
Dimension a(10)                   !      再用 DIMENSION 声明大小
REAL, DIMENSION(3,4) :: C, D
INTEGER IA(20,10), IB(-10:10, 20:40)
integer a(0:5)    ! a(0)~a(5)，共 6 个元素
integer a(-3:3)   ! a(-3)~a(3)，共 7 个元素
```

缺省下界为 **1**；也可声明为 0 或负数。**下标**用逗号分隔，可以是整型常量、变量、表达式甚至函数，但**必须在声明范围内**（越界是数组使用中最常见的问题）。引用的下标个数必须与维数一致。自定义类型也可以声明为数组：`type(person) :: a(10)`，元素用 `a(2)%height` 访问。

3.10.3 赋初值

```
integer A(5)
Data A /1,2,3,4,5/          ! A(1)=1 … A(5)=5
Data A /5*3/                ! * 表示重复：5 个 3
Data (A(I),I=2,4) /2,3,4/   ! 隐含 DO 循环：只给部分元素
Data ((a(i,j),i=1,2),j=1,2) /1,2,3,4/  ! 嵌套隐含 DO，先内后外
integer :: a(5) = (/ 1,2,3,4,5 /)      ! F90 构造器，括号和斜杠间不能有空格
integer :: a(5) = 5         ! 整体赋同一个值
```

易错：用构造器 `(/ /)` 赋初值时**不能**像 DATA 那样用隐含 DO 只给部分元素（每个元素都必须给值）；括号与斜杠之间不能有空格。

3.10.4 数组片段与整体操作（F90 特色）

```
a(3:5) = 5          ! a(3)=a(4)=a(5)=5
a(3:)  = 5          ! 从 a(3) 到结尾都设为 5
a(1:5:2) = 3        ! a(1)=a(3)=a(5)=3（第三个数字是步长）
a(1:10) = a(10:1:-1) ! 反转 a(1~10)
a(:) = b(:,2)       ! 形状要匹配：a(i)=b(i,2)
a = b + c           ! 整个数组相加：a(i,j)=b(i,j)+c(i,j)
a = b * c           ! 对应元素相乘（不是矩阵乘）
a = sin(b)          ! 逐元素函数
a = 5               ! 所有元素设为 5（F77 需要一层循环）
```

- **原则一**：等号两边使用的元素数目必须一样多；
- **原则二**：多个隐含循环时，**低维循环是内层**（先执行）；
- 数组整体赋值是**并行语义**：`a(1:9)=a(0:8)` 的结果是整体平移（/0,0,1,2,…,8/）；而 `DO i=1,9; a(i)=a(i-1); END DO` 是串行语义，结果是全部变成 0！要串行达到并行效果必须先 `b=a` 拷贝。

3.10.5 WHERE 与 FORALL

```
where( a < 3 )           ! F95：按逻辑判断挑选数组元素
  b = a                 ! 仅对 a<3 的位置执行
elsewhere
  b = 0
end where
! 单行：where(a<3) b=a
forall( i = 1:5 ) a(i) = i     ! 隐含循环式整体操作
forall( i=1:5, j=1:5, a(i,j)>0 ) a(i,j) = 1.0/a(i,j)  ! 带掩码
```

- WHERE 相当于对数组的 IF：模块内只能出现**设置数组**的语句，各数组必须**同维同大小**；可嵌套、可署名；
- FORALL 用隐含循环处理数组：`forall(triplet1, triplet2, …, mask)`；比 DO 循环的优势是**语句可按任意顺序执行**，便于并行优化；
- **关键区别**：**FORALL 里可以用 WHERE，WHERE 里不能用 FORALL**——WHERE 对整个数组做一个条件判断，FORALL 可对部分数组做多个条件判断；
- FORALL 块内多条语句**先执行完第一条的所有元素，再做第二条**。

3.10.6 存储规则：Column Major（按列存储）

数组不管声明成几维，在内存中都**连续存放**。Fortran 采用**按列存储**：多维数组**第一个下标变动最快，最右边的下标变动最慢**（与 C 的按行存储相反）。例：`A(3,3)` 在内存中的顺序是 A(1,1), A(2,1), A(3,1), A(1,2), A(2,2), A(3,2), A(1,3), A(2,3), A(3,3)。

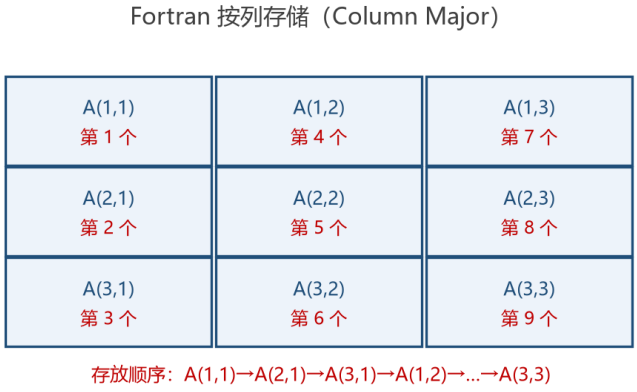


图 3.10-1　A(3,3) 的按列存储顺序：先走完第一下标（列），再换下一个下标记号

```
! 不推荐：a(i,j) 与 a(i,j+1) 在内存中不连续
DO i=1,n
  DO j=1,m
    a(i,j) = ...       ! 内层循环应该走内存连续方向
  ENDDO
ENDDO
! 推荐：a(j,i) 与 a(j+1,i) 相邻
do j=1,m
  do i=1,n
    sum = sum + a(i,j)
  end do
end do
```

要点：循环顺序应让**内层循环对应第一个下标**，访问内存连续的单元，程序更快。

3.10.7 动态数组 ALLOCATABLE

```
integer, allocatable :: a(:)       ! 声明可分配数组（维数用冒号占位）
write(*,*) "How many students:"
read(*,*) students
allocate( a(students) )             ! 运行期分配内存
! ...使用...
allocate( a(students), stat=error ) ! 检查分配是否成功
if ( error /= 0 ) ...               ! 0 成功，正数失败
if ( allocated(a) ) ...             ! 检查是否已分配
deallocate( a )                     ! 释放内存
deallocate( b, stat=error )
```

- F77 数组必须有明确的维数和大小；F90 引入动态数组解决“大小运行期才知道”的问题；
- **已被分配的数组不能再分配**，否则运行错误；只有 ALLOCATE 分配的空间才能被 DEALLOCATE 释放；
- 良好习惯：每个 `allocate` 都带 `stat=` 子句检查返回值，用完**始终** `deallocate`。

3.10.8 数组内在函数速查

函数	功能	函数	功能
`ALL(mask[,dim])`	指定维上是否全部满足 mask	`ANY(mask[,dim])`	指定维上是否有元素满足 mask
`COUNT(mask[,dim])`	满足条件的元素个数	`SUM(array[,dim][,mask])`	元素求和
`PRODUCT(array[,dim])`	元素连乘	`MAXVAL / MINVAL`	满足条件的最大/最小值
`MAXLOC / MINLOC`	最大/最小值的位置	`DOT_PRODUCT(a,b)`	两个向量的点乘
`MATMUL(A,B)`	**矩阵乘积**（二维）	`TRANSPOSE(matrix)`	转置二维数组
`SHAPE(source)`	返回数组形状	`SIZE(array[,dim])`	数组大小/某维长度
`LBOUND / UBOUND`	某维下界/上界	`RESHAPE(source,shape[,pad][,order])`	改变数组形状
`MERGE(t,f,mask)`	按 mask 组合两数组	`PACK(array,mask[,vec])`	压缩为向量
`UNPACK(vec,mask,field)`	解压为数组	`SPREAD(src,dim,n)`	复制扩展一维
`CSHIFT / EOSHIFT`	循环位移/带边界位移	`ALLOCATED(a)`	数组是否已分配

RESHAPE 可把一维数组改成任意形状，也可用于 Fortran（列主序）与 C（行主序）之间的数组序转换；元素总数必须一致。

3.11 函数与子程序

3.11.1 SUBROUTINE 子程序

```
program ex0801
  implicit none
  call message()        ! 用 CALL 调用
  stop
end
subroutine message()    ! 子程序定义
  implicit none
  write(*,*) "Hello."
  return                ! 返回调用处（可省略；若用 STOP 会终止整个程序！）
end
```

- 子程序是**具有特定功能的一段代码**，必须经 CALL 调用才执行（主程序自动执行）；
- 子程序与主程序地位相同：独立拥有自己的变量声明和行号，可以调用其他子程序，也可以**递归调用自己**；
- 气象数值模式（如 WRF）都把核心部分放进子程序，再经调用执行。

3.11.2 传址调用（call by reference）

核心概念：Fortran 传参是**传址调用**——调用时传出去的实参与子程序中接收的形参使用**相同的内存地址**。因此**子程序里修改形参，主程序里的实参同步改变**。

```
program main
  real :: a = 1
  call sub(a)       ! 传 a 的地址
  print *, a        ! 输出 5（不是 1！）
end
subroutine sub(b)
  real :: b
  b = 5
end subroutine
```

3.11.3 FUNCTION 自定义函数

```
program ex0807
  implicit none
  real :: a = 1, b = 2
  real, external :: add      ! 声明 add 是函数（external 建议不省略）
  write(*,*) add(a, b)       ! 直接用函数名调用，不用 CALL
  stop
end
real function add(a, b)      ! 推荐：返回类型写在 function 前
  implicit none
  real :: a, b
  add = a + b                ! 函数名即返回值变量
```

```
  return
end
```

- 与子程序的区别：函数调用**先要声明**（`real, external :: add`，F77 中 `external add` 单独写）；函数**返回一个数值**；
- 函数返回值可参与运算：`D = add(1.0,2.0) + add(2.0,3.0)`，可嵌套 `add(1.0, add(2.0,3.0))`；
- **不成文规定**：传给函数的参数只读不写；需要修改传入数据时用子程序；
- 函数或子程序名也可作为参数传递，声明时 `external`（自定义）或 `intrinsic`（库函数）不能省略。

3.11.4 数组与字符形参

- 数组参数传递的是**第一个元素的内存地址**，被调方按自己的声明解释连续内存——因此**形参、实参的个数、类型、次序必须完全一致**；
- 形参数组内存已存在，**大小可用变量指定**、甚至不指定：`integer :: num(*)` 为不定大小形参（无法越界检查，不推荐，未来标准将取消）；多维数组传参时**只有最后一维可以不指定大小**；
- 显式结构形参：把各维宽度也作为参数传入，如 `subroutine GetArray(a, dim1, dim2); integer :: a(dim1,dim2)`；
- 字符串形参可用**假定长度**：`character(len=*) :: string`（长度由实参决定）；
- **严重错误**：该传数组却传了标量，子程序会把变量及其后的内存当成大数组读写，极难调试。

3.11.5 SAVE 与变量生存周期

子程序里的变量默认**只在本次调用期间存活**，调用结束后消亡。加 SAVE 后变量与整个程序同寿命，**初值只设置一次**，下次调用保留上次的值：

```
subroutine count_calls()
  integer, save :: count = 0    ! 推荐写法：save 与声明同行
  count = count + 1
  write(*,*) count              ! 每次调用递增
end subroutine
```

注意：有些编译器即使不写 SAVE 也会记住变量值，为**可移植性**需要时务必显式加 `save`。COMMON 公用块的 SAVE 写成 `SAVE /blockname/`。

3.11.6 递归函数

```
recursive integer function fact(n) result(ans)
  implicit none
  integer, intent(in) :: n
  if ( n < 0 ) then
    ans = -1           ! 不合理的输入
  else if ( n <= 1 ) then
    ans = 1
  else
    ans = n * fact(n-1)   ! 递归调用自己
  end if
end function
```

- 只有加了 `recursive` 的函数/子程序才能递归调用；
- 递归的必要条件：每次调用时**局部变量使用不同的内存地址**（无 save 的局部变量每次调用独立）；
- 标准要求递归函数用 `result(ans)` 改名设置返回值（有些编译器接受不改名的写法）。

3.11.7 参数属性 INTENT（第十四讲）

F90 允许给形参加上**参数属性**，用 `INTENT` 命令规定参数是只读、只写还是可读写。不指定属性**不会影响运行结果**，但提倡使用——它能提高程序的可读性与可维护性，并帮助编译器查出误改参数一类的错误。

属性	含义	对实参的限制
`INTENT(IN)`	只用于**向过程提供数据**：执行期间不能被重新赋值，也不能变成未定义	可以是常数、变量、数组以及它们的**算术表达式**
`INTENT(OUT)`	只用于**把数据传回调用程序**：进入过程时其值未定义，过程内应重新赋值	**只允许是变量**，不能是常数或算术表达式
`INTENT(INOUT)`	既提供数据又传回数据	**只允许是变量**
缺省（未写 INTENT）	可读可写	同变量

```
subroutine div(a, b)
  implicit none
  integer, intent(in)  :: a   ! 只读：在子程序里重新赋值会“编译出错”
  integer, intent(out) :: b   ! 要输出：若在子程序中不重新赋值会“编译警告”
  b = a / 2
end subroutine
```

- **规则**：设置成只读的变量在过程内**重新设值 → 编译错误**；设置成要输出的变量**没有重新设值 → 编译警告**；
- **范围**：`INTENT` 属性**只对过程的形参有效**，用它声明局部变量或主程序变量都是错误的；
- 良好习惯：**始终为每个形参声明 intent**，可以防止过程偶然修改自己的输入参数。

3.11.8 过程接口 INTERFACE（第十四讲）

`INTERFACE` 是一段程序模块，用来**清楚说明所调用过程的参数类型、属性与返回值类型**。它写在主调程序的说明部分中（通常位于类型说明语句之前），内容是**被调用过程的说明部分**的拷贝，作用是通知编译系统如何正确地传递参数。一般情况下不需要特别说明用法；**模块中的过程已经自带接口**，用 `use` 引入后不必再写。

必须写接口的情形（考试常考）：

- 函数**返回值为数组**；
- **按名字（关键字）传递参数**、参数数目不固定（可选参数）；
- 参数是指针，或函数返回值为指针；
- 传递的参数是**不定结构数组**（assumed-shape 数组）；

- 函数前缀关键词是 ELEMENTAL；
- 调用过程用**超载赋值号**（子程序）或**超载操作符**（函数）；
- 字符型函数的结果长度不是常数、也不是假定长度 *。

```
interface
  function random10(lbound, ubound)    ! 函数接口
    implicit none
    real :: lbound, ubound
    real :: random10(10)               ! 返回值是个数组
  end function random10
  subroutine sub(a, b)                 ! 子程序接口
    implicit none
    integer :: a
    integer, optional :: b             ! b 可以不传入
  end subroutine sub
end interface
```

- 接口块以 INTERFACE 开始、END INTERFACE 结束，块内**只能取被调用过程说明部分，不允许出现任何可执行语句**；
- 接口体中**不能含有** ENTRY、DATA、FORMAT 与语句函数语句；接口块也**不允许出现在 BLOCK DATA** 程序单元中；
- 一个接口块中可以说明**多个**被调用过程，各自用自己的开始/结束语句定界，次序任意。

3.11.9 不定个数参数与按名传参（第十四讲）

F77 的函数参数个数必须固定；F90 用 **OPTIONAL** 声明“可以省略”的形参，用**内在函数 PRESENT** 检查该参数这次是否被传入（返回逻辑值），从而给缺省参数设定默认值：

```
subroutine sub(a, b)
  implicit none
  integer :: a
  integer, optional :: b        ! b 是可以省略的参数
  if ( present(b) ) then
    write(*, "('a=',I3,' b=',I3)") a, b
  else
    write(*, "('a=',I3,' b=unknown')") a
  end if
end subroutine
! 调用：call sub(1) 与 call sub(2,3) 都合法
```

按名传参（改变参数传递位置）：调用时用变量名直接代入，可以**不按形参顺序**传递。规则是“**前面按顺序写的部分要保持原次序，从某个参数开始用名字的部分可以任意排队**”：

```
subroutine sub(a, b, c)
  ...
end subroutine
! 三种合法写法：
call sub(1, 10, 100)          ! 全按顺序
call sub(1, 10, c=100)        ! 前两个按顺序，第三个用名字
call sub(c=100, b=10, a=1)    ! 全部用名字，次序任意
! 错误写法：
```

```
call sub(10, 1, c=100)         ! 前两个参数次序颠倒
call sub(1, 10, c=100, 200)    ! 一旦开始用名字，后面每个都要写成“名字=值”

real function func(x, a, b, c)   ! func(x) = a*x^2 + b*x + c，a、b、c 缺省为 0
  implicit none
  real :: x                        ! x 必须传入
  real, optional :: a, b, c        ! a、b、c 可以不传入
  real :: ra, rb, rc
  ra = 0.0; rb = 0.0; rc = 0.0
  if ( present(a) ) ra = a
  if ( present(b) ) rb = b
  if ( present(c) ) rc = c
  func = ra*x**2 + rb*x + rc
end function
! 调用：func(2.0, c=1.0) → 1.0 ； func(2.0, a=2.0, b=1.0) → 10.0
```

注意：调用这类参数数目不固定的过程时，**必须**先写出接口 `INTERFACE`（过程封装在 module 中则不必）。这种方法在过程可以接收很多参数时，能省去大量不必要的描述。

3.11.10 内部函数（CONTAINS）、PURE 与 ELEMENTAL

F90 的特殊函数类型有四种：**RECURSIVE（递归）**、**内部函数**、**PURE**、**ELEMENTAL**。内部函数把过程“归属于”某一个程序单元，只能用 `CONTAINS` 定义、只能被包含它的程序单元调用：

```
program ex0831
  implicit none
  integer :: n
  read(*,*) n
  write(*, "(I2,'! = ',I8)") n, fact(n)   ! 无需接口声明即可调用
contains                                  ! 后面开始写内部函数
  recursive integer function fact(n) result(ans)
    implicit none
    integer, intent(in) :: n
    if ( n <= 1 ) then
      ans = 1
    else
      ans = n * fact(n-1)
    end if
  end function fact
end program
```

- 内部函数**只能被包含它的程序单元调用**，别的过程不能调用它；主程序调用内部函数**不需要**声明接口；
- **PURE 函数（纯函数）**：在 function/subroutine 前加 `pure`。限制——函数的参数必须都是 `intent(in)`、子程序的每个参数都要有属性、**不能使用 SAVE**、内部命名的过程也必须都是 pure；
- PURE 函数中**不能使用 STOP、PRINT 及任何与输入输出有关的命令**（read、write、open、close、backspace、endfile、rewind、inquire 等），也**只能读取、不能改变全局变量**的值；
- **ELEMENTAL 函数（逐元函数）**：除 pure 的限制外，**所有形参与返回值都必须是标量**（不能是数组或带 pointer 属性）。它的价值在于配合 F90 的整数组语法：

```
elemental real function func(a)
  real, intent(in) :: a
  func = a * 2.0
```

```
end function
! 于是可以对整个数组一次操作：
a = func(a)          ! 等价于下面的循环
! do i = 1, 10
!   a(i) = func(a(i))
! end do
```

3.12 全局变量、模块与接口

3.12.1 COMMON 全局变量

```
program main
  implicit none
  integer :: a, b
  common a, b          ! a,b 是全局变量中的第 1、2 个变量
  a = 1; b = 2
  call ShowCommon()
end
subroutine ShowCommon()
  implicit none
  integer :: num1, num2
  common num1, num2    ! 按位置对应 a,b（名称无关！）
  write(*,*) num1, num2
end
```

- COMMON 是 F77 的全局变量：在不同程序块之间**共享同一块内存**；
- **关键**：按**声明的相对位置**对应，而不是按变量名对应——子程序里只用到第 6 个变量也必须把前 5 个都声明出来；
- EQUIVALENCE 是**同一程序块内**两个变量共享内存；COMMON 是**不同程序块之间**共享内存；
- DATA 不能给 COMMON 变量赋初值，必须用 **BLOCK DATA** 块数据单元；
- **有名公用块**：common /group1/ a 把变量分到不同公用区，一个变量不能同时出现在两个有名公用块中；
- 原则：共享数据少、使用程序少 → 用参数；共享数据大、程序多 → 用全局变量。

3.12.2 BLOCK DATA 块数据单元

```
block data
  implicit none
  integer a, b
  common a, b
  data a, b /1, 2/          ! 全局变量初值
end block data
```

类似子程序但**不需要调用、在主程序执行前即生效**；只能包含与声明有关的语句，不能有执行命令；其中不能出现 PARAMETER、ALLOCATABLE、AUTOMATIC、EXTERNAL、INTENT、OPTIONAL、PRIVATE、PUBLIC 等属性。

3.12.3 MODULE 模块与 INTERFACE 接口

模块用 `module … end module` 封装类型、数据与过程，其他程序单元用 `use` 引入——是现代 Fortran 取代 COMMON 的推荐机制。接口块 `interface … end interface` 显式声明过程的参数表，让编译器能检查实参与形参的匹配（第 5 次作业代码即为范例：

```
module typedef
  implicit none
  type student
    integer :: Chinese, English, Math
  end type student
end module

program main
  use typedef        ! 使用模块
  implicit none
  interface          ! 显式接口：声明 data_interp 的参数表
    subroutine data_interp(lat_in, lon_in, v_in, v_out)
      real, allocatable :: lat_in(:), lon_in(:)
      real, allocatable :: v_in(:,:)
      real :: v_out(360,180)
    end subroutine data_interp
  end interface
  call data_interp(lat, lon, v, out)
end
```

3.13 文件操作

3.13.1 OPEN 打开文件

```
open(unit=10, file='hello.txt')   ! 打开文件，代号 10
write(10,*) 'hello'               ! 写入代号为 10 的文件
close(10)
```

OPEN 参数	取值与含义
`UNIT=number`	文件代号（正整数，常量或变量）；**避开 1、2、5、6**（系统保留：5 键盘输入、6 屏幕输出）
`FILE='filename'`	文件名；**尽量避免中文名**
`FORM=`	`'formatted'` 有格式（文本文件，默认）丨`'unformatted'` 无格式二进制丨`'binary'` 二进制流
`STATUS=`	`'new'` 文件不存在新建丨`'old'` 文件已存在丨`'replace'` 存在则覆盖丨`'scratch'` 临时文件（可不取名，程序结束自动删除）丨`'unknown'` 编译器决定（默认）
`ACCESS=`	`'sequential'` 顺序记录（默认）丨`'direct'` 直接记录（可任意指定位置读写）丨`'stream'` 流式访问（F2003）
`RECL=length`	顺序文件：一次读取的数据容量；直接记录文件：**每个记录的长度**（文本 1 字符 = 1 byte；二进制由编译器定义，

OPEN 参数	取值与含义
	一般 1 或 4 bytes）
POSITION=	'ASIS' 打开时读取位置不变（默认）｜'REWIND' 移到开头｜'APPEND' 移到结尾（F90 新增）
ACTION=	'READ' 只读｜'WRITE' 只写｜'READWRITE' 可读写（默认）

3.13.2 READ / WRITE 读写与位置控制

```
write(10, "(A20)") 'Good morning.'   ! 写入文件
rewind(10)                           ! 读写位置倒回文件开头
read(10, "(A20)") string             ! 从文件读出
write/read 参数：UNIT、FMT、NML=namelist、REC=record（直接存取记录号）、IOSTAT、ERR、END、
ADVANCE、SIZE
```

3.13.3 INQUIRE 查询与 CLOSE

```
character(len=20) :: filename = 'ex0903.f90'
logical alive
inquire(file=filename, exist=alive)   ! 文件是否存在
if ( alive ) write(*,*) filename, " exist."
```

INQUIRE(UNIT=/FILE=, EXIST=, OPENED=, NUMBER=, NAMED=, ACCESS=, SEQUENTIAL=, DIRECT=, FORM=, FORMATTED=, UNFORMATTED=, RECL=, …) 查询文件是否存在、是否已打开、访问方式、记录长度等。用完文件必须 close(unit)，以便其他程序使用该文件。

3.13.4 顺序、直接访问与二进制文件

类型	特点
顺序文件	只能**从头一步步**读写，不能任意跳到某个位置；要改变位置只能逐步进退或 REWIND 回开头
直接访问文件	access='direct' 与 recl **不能省略**；rec= 指定记录号即可随机读写；文本格式下 RECL = 一行字符数（MS 系统中每行行尾有 2 个不可见符号，RECL 要 +2）
二进制文件	无格式文件：保存的是**内存中数据的原始格式**，体积小、读写快；直接读取时 rec 的大小单位随编译器变化（1 或 4 bytes）；form='unformatted' 是默认但建议显式写出

3.13.5 NAMELIST：成组变量的读写（第十五讲）

NAMELIST 把一组相关变量封装在一起，读写时只要在 NML= 字段写上该组的名字即可，程序会**按变量名自动对应**，不必关心顺序：

```
program ex0919
  implicit none
  integer :: a, b, c
```

```
  namelist /na/ a, b, c      ! 声明（必须写在执行语句之前，且必须取名）
  read (*, nml=na)
  write(*, nml=na)
end program
! 输入形式：&na a=1 b=2 c=3 /
```

- 可以不填所有变量、**可以不按顺序**输入；同名变量可以重复输入，只取**最后一次**的值；同一个 NAMELIST 字段在文件里多次出现时，只读取**第一次**的设定值；
- 所有变量应**先赋初值**——有的编译器默认初值为 0，有的会得到意想不到的值；
- 从文件读取时会**自动从当前位置向下**寻找存放 namelist 的地方；NAMELIST 一般用于文件数据输入，很少用于键盘输入。

3.13.6 内部文件与动态格式（第十六讲）

内部文件（Internal File） 就是字符串变量：用“写文件”的方法把数据写进一个字符串，也可以用“读文件”的方法从字符串里读出数据，这是把数值与字符串互相转换、拼接输出内容的常用手段。

```
character(len=40) :: line
integer :: year = 2026
real :: wind = 12.5
write(line, "('年份:',I4,' 风速:',F5.1,' m/s')") year, wind   ! 数值 → 字符串
read(line, *) y                                                ! 字符串 → 数值
! 动态改变输出格式：把格式串放进字符变量
character(len=20) :: fmt = '(F8.2)'
write(*, fmt) wind
```

3.13.7 顺序文件读写实战与注意事项（第十五讲）

- 顺序文件读写**不能任意跳到某个位置**，只能从头一步步向下；要改变位置只能逐步进退，或直接 `rewind` 回文件开头；
- 读取循环的常用写法：`read(fileid, "(...)", iostat=error) ...`，然后 `if ( error /= 0 ) exit` 在文件结束时跳出（`iostat > 0` 出错、`= 0` 正常、`< 0` 文件终了）；
- `ADVANCE='yes'/'no'` 控制文本顺序文件每次读写后**是否自动向下移一行**（默认 yes）；用 `advance='no'` 时可用 `SIZE=count` 取得本次读写的字符数；
- 位置控制命令：`BACKSPACE` 退回一步、`ENDFILE` 把当前位置变成文件结尾、`REWIND` 倒回文件开头；
- **注意**：**文件的代号码在整个程序中是共享的**，不同函数可以用同一代号读写同一个文件；程序可以同时打开多个文件，但操作系统会限制一个程序能打开的文件数量——**不要一次同时打开很多个文件**；程序用完文件后要 `close`，以便多任务环境中其他程序能使用该文件。

3.14 指针（POINTER）

3.14.1 基本概念：指针、目标与关联（第十七讲）

指针变量保存的是目标数据的内存地址。读取一个指针变量要分两步理解：(1) 取出指针中所保存的内存位置；(2) 到这个内存位置读取数据。指针中内存地址的来源有两种：**记录其他非指针变量的内存位置**，或**程序执行中动态配置一块内存**。

```
integer, target  :: a = 1, b = 2   ! target：声明一个可以当成指针目标的变量
integer, pointer :: p               ! 声明一个可以指向整数的指针
p => a                             ! 把指针 p 指到 a
write(*,*) p                       ! 输出 1
p => b                             ! 指针可以随时重新设置它的指向
write(*,*) p                       ! 输出 2

allocate(p)                        ! 配置一块可以存放 integer 的内存给 p
p = 100                            ! 拿到内存后可以像一般整数一样使用
deallocate(p)                      ! 释放内存；再 p => a 改变指向
```

写法	作用
`integer, target :: a`	声明**可以当成指针目标**的变量（被指向者必须带 TARGET 属性）
`integer, pointer :: p`	声明指针变量；`integer, pointer :: a(:)` 声明**指针数组**（只说明维数）
`p => a`	建立关联（把 p 指向 a）；此后 p 与 a 是同一块内存
`associated(p)` / `associated(p, a)`	检查指针**是否已设置指向** / 是否指向指定的目标 a
`nullify(p)`	F90：把指针设置成**还没有指向任何内存地址**
`integer, pointer :: p => null()`	F95：`null()` 返回一个不能使用的内存地址，可在声明时初始化指针

- **严重易错**：使用指针**之前一定要先设置好指针的目标，编译器不会检查**；未初始化就使用的指针会读写到不可预料的地址（用 `associated` 判断、用 `nullify/null()` 初始化可避免）；
- 指针可以声明成**任何数据类型**，包括 `type` 自定义类型；但**不管指向哪种类型，指针变量本身占用的内存都一样大**（32 位计算机 4 bytes），因为它记录的就是内存地址；
- 如果指针指向的是 `allocate` 分配的内存，**重新改变指向前**要先处理这块内存（交给别的指针或 `deallocate`），否则这块内存会被白白浪费。

3.14.2 指针数组与数组片段（第十七讲）

```
integer, pointer :: a(:)              ! 指针数组：只用冒号说明维数
integer, target  :: b(5) = (/ 1,2,3,4,5 /)
a => b             ! a(1:5) 依次对应 b(1:5)
a => b(1:3)        ! a(1)=>b(1), a(2)=>b(2), a(3)=>b(3)
a => b(1:5:2)      ! a(1)=>b(1), a(2)=>b(3), a(3)=>b(5)（步长 2）
a => b(5:1:-1)     ! 反向：a(1)=>b(5) … a(5)=>b(1)
allocate( a(5) )   ! 也可以 allocate 一块内存，把指针当动态数组用
a = (/ 1,2,3,4,5 /)
deallocate( a )    ! allocate 得到的内存要记得归还

integer, pointer :: a(:,:)            ! 二维指针数组
integer, target  :: b(3,3,2)
a => b(:,:,1)              ! 方便地取高维数据中的某一层
a => b(1:3:2, 1:2, 2)      ! a(1,1)=>b(1,1,2), a(2,1)=>b(3,1,2) …
```

程序中需要**反复使用数组的一小部分**时，声明一个指针来代表这部分数组最方便，代码也更清晰。**区别：动态数组（allocatable）在过程结束后会自动释放内存；指针数组不会自动 deallocate，必须自己用 DEALLOCATE 释放**。

3.14.3 指针与函数（第十七讲）

- 指针变量同样可以作为参数在过程之间传递，也可以作为函数的返回值；
- 把指针传递给函数时**必须声明该过程的 INTERFACE**；函数返回值若为指针，也必须定义接口；
- 指针形参**不需要**写参数属性 `INTENT`；
- 如果过程封装在 `MODULE` 中，就等于已经写好使用接口，调用时不必再写；
- 指针**提高效率**的原理：交换两个 `type(person)` 类型（约 28 bytes）的数据要移动 84 bytes，若改用指针，只需要交换 4 bytes 的地址（3×4 = 12 bytes）。

```
module func
contains
  function getmin(p)          ! 返回“最小元素”的指针
    implicit none
    integer, pointer :: p(:)
    integer, pointer :: getmin
    integer i, s, min
    s = size(p, 1)
    min = 2**30               ! 先设一个很大的值
    do i = 1, s
      if ( min > p(i) ) then
        min = p(i)
        getmin => p(i)        ! 让返回值指向最小元素
      end if
    end do
  end function
end module
! 排序应用：把排序子程序的形参写成 type(pperson)（其中 p 是指向 person 的指针），
! 选择排序时交换的只是 4 bytes 的指针，而不是整个 person 结构
```

3.14.4 指针的高级应用：链表（第十七讲）

把“数据 + 指针”做成**结点**再首尾相连，就得到**串行（链表）**结构：**单向串行**每个结点只记录下一条数据的地址；**双向串行**同时记录上一条与下一条的地址；首尾相接则成为**环状串行**。链表的优点是**插入与删除只需改动几个指针**，不必像数组那样搬动大量数据。

```
module typedef
  implicit none
  type :: datalink
    integer :: i
    type(datalink), pointer :: prev    ! 指向上一条数据
    type(datalink), pointer :: next    ! 指向下一条数据
  end type datalink
end module typedef

program ex1013
  use typedef
  implicit none
  type(datalink), target  :: node1, node2, node3
```

```
  type(datalink), pointer :: p
  node1 = datalink(1, null(), node2)   ! 首结点：prev 空，next 指向 node2
  node2 = datalink(2, node1 , node3)
  node3 = datalink(3, node2 , null())
  write(*,*) "照顺序输出"
  p => node1
  do while (.true.)
    write(*,*) p%i
    if ( .not. associated(p%next) ) exit   ! next 为空表示到表尾
    p => p%next
  end do
  write(*,*) "反过来输出"
  p => node3
  do while (.true.)
    write(*,*) p%i
    if ( .not. associated(p%prev) ) exit
    p => p%prev
  end do
end program
```

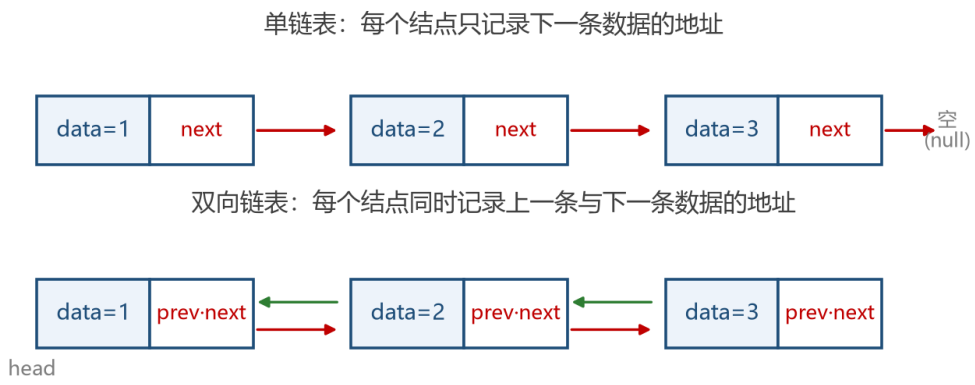


图 3.14-1　单链表与双向链表：每个结点由数据字段与指针字段构成，插入与删除只需改动相邻结点的指针

- 遍历用 `do while(.true.)` + `associated(p%next)` 判断是否到表尾，`p => p%next` 依次前进；
- **插入**：让新结点的 next 指向原来的后继，再让前一个结点的 next 指向新结点（顺序不能颠倒，否则会丢失后续数据）；**删除**：让前一个结点的 next 直接跳过被删结点，再释放该结点内存；
- 配合指针数组排序（3.15.1 的指针排序思想）：把所有结点指针放进数组，排序时只交换指针，效率远高于搬动数据。

3.15　数据结构与算法（教材第 15 章）

3.15.1　排序

排序 = 把一组数据按某种规则（从小到大/从大到小）排列。任何排序过程都包含两类基本操作：**元素值的比较**与**元素值的交换**。以下以数列 {3,2,5,1,4} 升序为例。

方法	原理	效率评价
冒泡排序 Bubble Sort	每轮把相邻两数比较、逆序则交换，一轮把**最大值沉到末尾**；N 个数共扫描 **N−1 轮**，每轮比	直观；每轮都能看到初步结

方法	原理	效率评价
	较对数递减	果
选择排序 Selection Sort	每轮在剩余数据中挑出**最小值**放到已排好的队尾：第 1 轮挑最小放第 1 位，第 2 轮挑次小放第 2 位…	直观；数据量大时效率不高
Shell 排序	令 K=N/2，把间隔 K 的元素分组比较交换（交换后还要**往回**间隔 K 继续比）；K 每次减半，直到 K=0	“乱中有序”，直到最后一轮才确定位置
快速排序	以第 1 个数为**键值 K**，L 从前往后找 ≥K、R 从后往前找 <K，交换并继续，直到 L>R 后交换 A(1) 与 A(R)；K 归位后**递归**排序左右两组	**目前最快**，用递归实现

综合效率排名（数据量大时）：**快速排序 > Shell 排序 > 冒泡排序 > 选择排序**。该排名不是绝对的：数据量小时各种方法几乎一样，选最熟悉的方法即可。

3.15.2 搜索

方法	原理与代价	适用条件
顺序搜索	从头到尾逐个比较；找 5 要 5 次（最坏 N 次）	任何数据都可；最简单
二分搜索	取中间值 M 与键 K 比：K>M 取后半段、K<M 取前半段，反复折半；找 5 只要 **3 次**（复杂度 $O(\log_2 N)$）	**必须已排序**；数据量大时优于顺序
散列搜索 Hashing	设计散列函数 hash(K) 直接算出数据在表中的位置，比较一次即可判断；冲突（不同 K 同位置）时向后挪一位或用链表	散列函数设计得好时**几乎一次比较**即中

散列搜索示例：数据在 0~100 间，声明 100 元素数组，hash(D)=D 直接放 array(D)（空间换时间）；改小表时 hash(K)=K/10+1，冲突后向后挪位（线性探测）。若数据不存在，小散列表要扫完整个表才能确认；配合**链表**存放同 hash 值的数据可大幅改善。

3.15.3 堆栈（Stack）

堆栈是**后进先出（LIFO）**的数据管理规则：“越早得到的数据越晚输出，越晚得到的数据越早输出”，像堆报纸一样。用数组 + 栈顶指针即可实现：`push` 把数据放栈顶、`pop` 从栈顶取出。典型应用：把输入的一串数据**反向输出**。

3.15.4 树状结构与二叉树

二叉树是链接成树枝状的结构：**左枝保存的数值永远小于右枝**。插入规则：从根开始，新数大于当前节点则向右、小于则向左，直到找到空位；相等则重复计数 +1。**中序遍历**（先左枝、再当前、后右枝）即可**由小到大输出**全部数据——建树的过程同时完成了排序。用指针 + 递归实现：

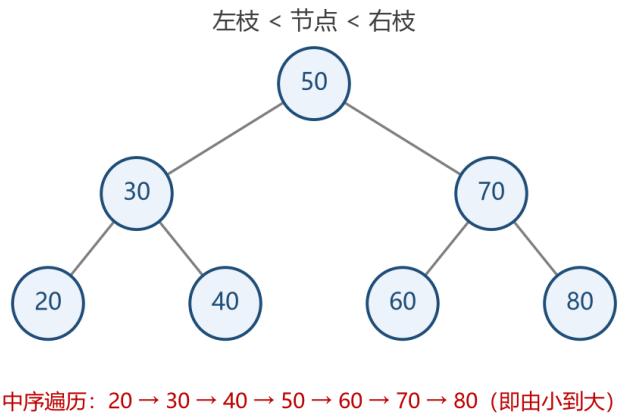


图 3.15-1　二叉树：任意节点的左枝数值小于该节点、右枝数值大于该节点；中序遍历即得升序序列

```
recursive subroutine show_tree(show)   ! 中序遍历（从小到大）
  if ( associated(show) ) then
    call show_tree( show%left )     ! 先左枝
    call show_data( show )          ! 再当前
    call show_tree( show%right )    ! 后右枝
  end if
end subroutine show_tree
```

3.16　综合实战：双线性插值（期中大作业）

应用背景：把气象数据插值到不同空间分辨率是大气科学科研最常见的操作。Matlab、NCL 自带插值函数，但对**两极的处理**和**经度周期边界**存在问题，因此课程要求从底层代码实现，深入理解插值过程。

3.16.1　问题与数据

- **GFDL-ESM2M** 数据：经度 2°、纬度 2.5° 的规则网格（经纬度是矩形格点的中心），插值到 1°；
- **FGOALS-s2** 数据：高斯网格（纬度不均匀），同样插值到 1°；
- 两个文件均为 CMIP5-RCP8.5 试验 2100 年 12 月的近地面风速（sfcWind），格式：`"LAT","LON","SFCWind (m/s)"` 三列。

3.16.2　双线性插值原理

双线性插值 = 在两个方向各做一次线性插值：已知 f 在矩形四角 $Q_{11}=(x_1,y_1)$、$Q_{12}=(x_1,y_2)$、$Q_{21}=(x_2,y_1)$、$Q_{22}=(x_2,y_2)$ 的值，求点 P=(x,y) 的值。**先 x 方向**在上下两边插出 R_1、R_2，**再 y 方向**在 R_1、R_2 间插出 P。

$$f(x,y_1) \approx \frac{x_2-x}{x_2-x_1} f(Q_{11}) + \frac{x-x_1}{x_2-x_1} f(Q_{21}) = f(R_1)$$

$$f(x,y_2) \approx \frac{x_2-x}{x_2-x_1} f(Q_{12}) + \frac{x-x_1}{x_2-x_1} f(Q_{22}) = f(R_2)$$

$$f(x,y) \approx \frac{y_2-y}{y_2-y_1} f(R_1) + \frac{y-y_1}{y_2-y_1} f(R_2)$$

两个方向插值的先后顺序不影响结果。若取单位正方形（四角坐标 (0,0)、(1,0)、(0,1)、(1,1)），公式化简为：

$$f(x,y) = f(0,0)(1-x)(1-y) + f(1,0)x(1-y) + f(0,1)(1-x)y + f(1,1)xy$$

当 z 在 x、y 方向上有单调性时，该方法也可做**外插**（求正方形以外的点）。

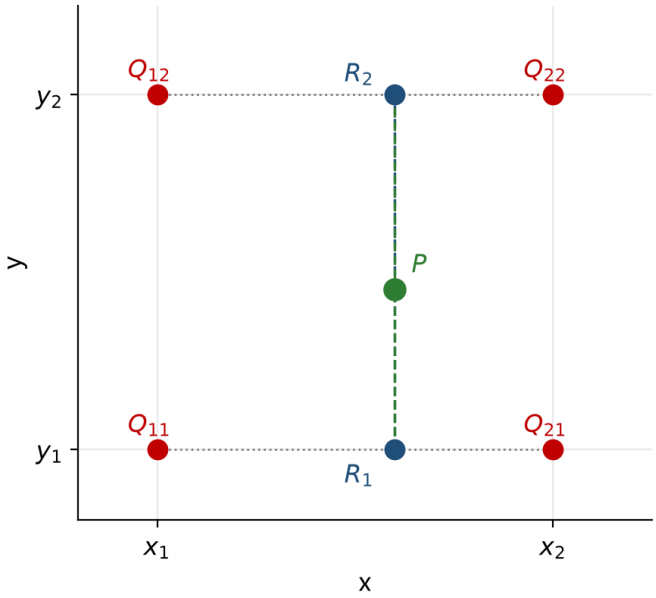


图 3.16-1　双线性插值：红点 Q_{11}、Q_{12}、Q_{21}、Q_{22} 为已知格点，蓝点 R_1、R_2 为 x 方向插值结果，绿点 P 为待求点

3.16.3　程序结构与关键处理

```
! 主程序：变量命名、调用子程序；子程序：读取、插值、输出
interface
  subroutine data_read(file_name, wind_srf_ori)      ! 读文件（跳过表头行）
  subroutine data_interp(wind_srf_ori, wind_srf_interp)
  subroutine data_output(file_name, wind_srf_interp)  ! 按原格式写出
end interface
! data_interp 核心：找包围目标点的 4 个格点
call lat_search(lat_out(j), lat_in, lat1, lat2)  ! 纬向最近两点
call lon_search(lon_out(i), lon_in, lon1, lon2)  ! 经向最近两点
fq11 = (x2-x)/(x2-x1) * v_in(lon1,lat1)   ! 先 x 方向
fq21 = (x-x1)/(x2-x1) * v_in(lon1,lat2)
fp1  = (y2-y)/(y2-y1) * (fq11+fq21)       ! 再 y 方向
v_out(i,j) = fp1 + fp2
```

- **经度周期边界**：东经 180° 与西经 180° 是同一地点——lon_search 中当目标点在 lon_in(1) 之前或 lon_in(末) 之后时，取“最后一格 + 第一格”作为包围点；
- **两极外推**：假设高于数据最高纬度的风速不变——最外圈（±89.5°）直接用最近一圈的纬度值（用同一条纬度上的线性插值，不参与纬向插值）；
- **通用化**：lat_in/lon_in 用可分配数组 + size()，程序自动适应任意分辨率输入；
- **lat_search 技巧**：`lat_distance = lat_in - targetpnt`，找出乘积 `lat_distance(i)*lat_distance(i+1) < 0` 的那对相邻格点，即包围点。

3.17　编程规范与调试建议

- **命名**：不使用 a、b、c 等意义不明的变量名；变量用途与程序逻辑用注释写清楚，**注释必须用英文**（作业强制要求）；
- **排版**：声明、运算、输出等不同功能区之间用空行隔开；缩进对齐；
- 每个 `allocate` 带 `stat=` 检查，用完 `deallocate`；
- **调试期打开编译器的越界检测选项**，发布前关闭（越界检测会拖慢运行）；
- 数组使用前必须初始化；未初始化的数组结果不可预测，且随处理机不同而异；
- 使用有名常数（PARAMETER）声明数组大小，便于统一修改；
- 作业提交规范：压缩包命名为 学号_姓名_第几次作业，含源代码（.f90）与运行结果截图。

3.18　面向对象编程：MODULE 的深入用法（第十八讲 a）

3.18.1　结构化与面向对象

结构化与**面向对象**是目前程序设计时最常用的两种编程概念。结构化程序的特色是**层次分明**：可以分成不同的程序模块，同一模块的代码按自上而下的顺序一行行执行。**面向对象**做的是程序代码与数据的**封装**：封装后区分**公用数据**与**私有数据**（为了安全，有些数据不应让外界使用），函数同样可以封装成公用与私有；另一个重点是**程序代码的重复使用**——用 MODULE 就能以类似**继承**的方法重复使用函数代码。

3.18.2　MODULE 的结构与 PUBLIC / PRIVATE

MODULE 用 `module … end module` 封装程序模块，把功能相关的函数与变量放在一起。它可以做四件事：

- 在 module 里**声明变量**（声明时加 `save`，功能上等于全局变量）；
- 在 module 里**定义自定义类型**，再 `use` 引用；
- 在 module 里**编写函数**（`contains` 之后），通过 `use` 引用；
- module 里的函数**可以直接使用同一 module 中声明的变量**。

模块中封装的数据可以用 **`PUBLIC`（公用）/ `PRIVATE`（私有）** 分别指定是否对外开放：

```
module bank
  implicit none
  integer :: money = 1000000
  integer :: fileid = 10
  private money, fileid    ! 这两个变量不对外公开
  private TimeLog          ! 这个函数不对外公开
contains
  subroutine TimeLog()     ! 往 log 文件写入现在的时间
    ...
  end subroutine
  subroutine LoadMoney(name, num)   ! 公用：可被外部调用
    ...
  end subroutine
end module
```

```
! 也可以整体设置默认，再单独开口子：
module A
  implicit none
  private           ! 没有特别指明的都不对外公开
  public c          ! 只公开变量 c
  integer a, b, c
end module
module B
  implicit none
  public
  private c         ! 只隐藏 c，其余公开
  integer a, b, c
end module
```

- 只有用 USE 命令才能让模块外的程序使用模块内的数据与函数；
- **注意**：在过程里声明局部变量时，**名字不应与 use 引入的变量同名**，否则会编译错误；
- 用 module 在过程之间传递大量数据时，应**始终在模块中包含 save 语句**，以确保不同过程访问数据时数据保持稳定；use 语句应放在 program/subroutine/function 语句后面的**第一个非注释语句**位置。

3.18.3 USE 的用法与“继承”

```
use A                     ! 引入模块 A 的全部公用内容
use A, aa => va           ! 把 A 中的变量 va 改名成 aa 使用
use A, only : vc          ! 只引入其中的变量 vc
use A, only : c => vc     ! 只引入 vc，并改名为 c
! 一个程序可以同时 use 多个模块，只要多写几个 use 即可
```

在 module A 中使用 module B，可以想象成 **module A 继承了 module B 的数据和函数**（第 3.12.3 节的 module shoot use constant, typedef 就是这种用法）。**限制**：**继承只对模块中的公用数据和函数有效，私有数据不能继承**。

3.18.4 同名函数重载（OVERLOAD）

程序中可以同时拥有多个**名称相同、但参数类型或数目不同**的函数，编译器会根据实参自动决定调用哪一个——这叫**重载**。做法是在 MODULE 中用 INTERFACE 定义一个**虚拟的函数名**，再用 module procedure 列出候选函数：

```
module MA
  implicit none
  interface show                    ! 虚拟的函数名 show
    module procedure show_int       ! 候选函数 1
    module procedure show_character ! 候选函数 2
  end interface
contains
  subroutine show_int( n )
    integer, intent(in) :: n
    write(*, "('n=',I3)") n
  end subroutine
  subroutine show_character( str )
    character(len=*), intent(in) :: str
    write(*, "('str=',A)") str
  end subroutine
```

```
end module
! 调用：call show(1) 自动选择 show_int；call show("FORTRAN 95") 自动选择 show_character
! 另一个例子：interface getx / module procedure getx1, getx2，
! call getx(1.0,2.0) 调用二元一次方程求根，call getx(1.0,3.0,2.0) 调用二次方程求根
```

3.18.5 自定义操作符

用 type 定义的自定义类型数据**不能直接用 + - * / 做数值计算**，这时可以通过 INTERFACE **自定义操作符**完成运算：

```
module mytype
  implicit none
  type vector
    real :: x, y
  end type vector
  interface operator(+)                          ! 重载加号
    module procedure add_vector
  end interface
  interface assignment(=)                        ! 重载赋值号
    module procedure assign_vector
  end interface
contains
  function add_vector(a, b) result(c)
    type(vector), intent(in) :: a, b             ! 候选函数必须明确写出每个参数的 INTENT
    type(vector) :: c
    c%x = a%x + b%x
    c%y = a%y + b%y
  end function
  subroutine assign_vector(a, b)
    type(vector), intent(out) :: a
    type(vector), intent(in)  :: b
    a = b
  end subroutine
end module
```

- 把运算符号当虚拟函数名时，INTERFACE 后面要先接 OPERATOR，再用 () 把运算符包起来，如 `interface operator(+)`；
- 接口中等待候选的函数，**必须明确写出每一个参数的 INTENT 属性**；
- 重载赋值号写作 `interface assignment(=)`；一旦使用超载赋值号或操作符，调用方就需要接口（见 3.11.8）。

3.19 编译器的高级使用与程序优化（第十八讲 b）

3.19.1 编译器、链接器与链接库

部件	职责
编译器（Compiler）	把程序代码转换成目标文件（*.obj）或执行文件（*.exe）的工具，如 `fl32.exe`、`pgf90`、`f77`、`ifort`
链接器（Link）	把目标文件链接成可执行文件（*.exe），或形成链接库（*.a、*.lib）；**一般编译器会自动调用 LINK**

部件	职责
链接库（Library）	预先编译好 Fortran 函数并存放在库文件中，链接时取出所需函数

- **静态链接库 `*.lib`**：事先把要用的函数编译成机器码保存在 lib 文件中，链接时把**函数的机器码复制一份**放入执行文件；若链接库中已经编写了 module，使用时还需要 `*.mod` 文件；
- **动态链接库 `*.dll`（Windows）/ `*.a`（Unix）**：编译时**不复制**函数的机器码，只在执行文件中记录“要调用的函数在哪个库文件里”；程序执行时操作系统再把库中的函数取出来用；
- 动态链接库的优点：(1) **容易更新**——库与执行文件分散在不同文件中，不必重新编译执行文件就能使用新版链接库；(2) **方便 Fortran 程序给其他语言使用**；
- 编写 dll 时要在代码中说清楚哪些函数可以被外界调用：`!DEC$ ATTRIBUTES DLLEXPORT :: SUB`——这类**隐藏在批注中的命令是编译功能选项**，每个想被外界调用的函数都需要它才能对外公开。

3.19.2 程序代码优化的策略

程序的好坏通常取决于**执行速度**与**内存使用量**，当前数值计算更重视**执行速度**。优化的具体做法：

策略	说明与示例
使用好的算法	算法是解决某个问题的特定程序方法。计算 1+2+3+…+N，用循环累加远不如梯形公式 `s = (n+1)*n/2` 快。**对程序优化最有帮助的就是先进算法**
避免重复计算	二次方程求根时先算公共量：`sqrt_d = sqrt(b*b-4*a*c)`、`a_2 = 2.*a`，再代入 `x1=(-b+sqrt_d)/a_2`、`x2=(-b-sqrt_d)/a_2`
表达式的选择	`2*A = A+A`；`A**2 = A*A`；`X**2+2*X+3 = ((X+2)*X)+3`（减少乘幂与重复乘法）
整数与浮点数的选择	能用整数计算尽量用整数计算
访问速度	PARAMETER 常量的访问速度比较快；**数组的访问速度比变量慢，维数越大越慢**
利用 CACHE	程序执行时尽量避免跳转式使用内存；相邻代码中使用**内存地址相邻**的变量，命中 CACHE 的概率更高
减少程序代码的跳转	IF、SELECT CASE、DO 循环以及调用函数都会导致跳转与转向操作，应减少不必要的分支

```
! 数组访问配合内存排列顺序（内层循环走第一个下标）：
do j = 1, n
  do i = 1, m
    a(i, j) = a(i, j) * 2     ! 计算更快
  end do
end do
```

提醒：这些策略在实际使用时不必特别刻意（例如 `5*A` 不必写成 `A+A+A+A+A`），程序里不可避免地要用数组与循环，尽管放心使用，**只要注意数组访问尽量配合内存的排列顺序**即可。

3.19.3 Fortran 调用 C 程序

```
subroutine SUBA(a)
!DEC$ ATTRIBUTES C, ALIAS: '_sub1' :: SUBA
!DEC$ ATTRIBUTES REFERENCE :: a
!DEC$ ATTRIBUTES VALUE :: a
  ...
end subroutine
```

- `!DEC$` 是**跟编译器通信**的编译指令（隐含在批注中）；
- `ATTRIBUTES C` 说明这个函数是用 C 语言编写的；`ALIAS: '_sub1'` 指定要调用的**编译过的 C 函数名**；`SUBA` 是在 Fortran 中调用它所用的名字，**可以另取**，不必与原 C 函数同名；
- `ATTRIBUTES REFERENCE` 表示传递参数时传**地址**（对应 C 函数的指针形参）；`ATTRIBUTES VALUE` 表示传**数值**；经过 `ATTRIBUTES C` 设置后默认按值传递，因此 `VALUE` 可以省略。

3.20 课程各讲内容索引与作业清单

3.20.1 第 1~19 讲内容索引

讲次	课件主题	本手册对应章节
第 1 讲	Fortran 语言的历史、如何学好 FORTRAN、编译器使用	3.1、3.2
第 2 讲	Fortran 程序设计基础	3.3
第 3~5 讲	Fortran 输入输出（IO）及声明（一）～（三）	3.4、3.5、3.7
第 6~7 讲	流程控制与逻辑运算（一）（二）	3.8
第 8~9 讲	循环（一）（二）	3.9
第 10~11 讲	数组（一）（二）	3.10
第 12~13 讲	函数（一）（二）	3.11.1~3.11.6
第 14 讲	函数（三）：参数属性、使用接口、不定个数参数、改变参数传递位置、特殊函数类型	3.11.6~3.11.10、3.12.3
第 15 讲	文件（一）：OPEN / READ / WRITE / INQUIRE、顺序文件	3.13
第 16 讲	文件（二）：直接访问文件、二进制文件、内部文件、NAMELIST	3.13.4~3.13.6
第 17 讲	指针：基本概念、指针数组、指针与函数、链表	3.14
第 18 讲（a）	MODULE 及面向对象：PUBLIC/PRIVATE、继承、重载、自定义操作符	3.12.3、3.18
第 18 讲（b）	编译器的高级使用：链接器与链接库、程序代码优化、Fortran 调用 C	3.19
第 19 讲	教材第 15 章 数据结构与算法：排序、搜索、堆栈、树状结构	3.15

讲次	课件主题	本手册对应章节
期中大作业	气象数据双线性插值（含经度周期边界与两极外推）	3.16

3.20.2 作业清单与提交规范

- **作业通用规范**（《作业要求》）：不可以使用 a、b、c 等意义不明的变量名；所声明变量的用途与程序逻辑要用注释写清楚，**注释不可以使用汉语**；不同功能区（声明、运算、输出）之间用空格隔开；程序缩进要**对齐**；
- **提交方式**：两周内把压缩包提交到指定邮箱，压缩包需包含**程序运行结果截图**与**源代码（.f90 文件）**，命名格式为 学号_姓名_第几次作业；
- **第十四讲作业**：① 利用递归函数计算等差数列 4+8+12+…+100；② 编写函数计算输入两个数的最大公因子；③ 编写程序，既能求四边形周长（A+B+C+D）也能求三角形周长（A+B+C），并规定当 D 不出现时置 D 值为零（提示：用 `OPTIONAL` + `PRESENT`）；
- **期中大作业（双线性插值）**：把 GFDL-ESM2M（2°×2.5° 规则网格）与 FGOALS-s2（高斯网格）的近地面风速插值到 1° 分辨率，以与输入文件相同的格式输出并提交源代码，截止日期 12 月 8 日；评分点包括**经度上的周期性、纬度上的极点处理、程序能否处理任何分辨率**；
- 作业代码可直接参考第 3.16.3 节的程序结构（主程序只负责变量命名与调用，子程序分别负责读取、插值、输出）。

附录

附录 A　易错点与重点标注清单（教材手写笔记摘录）

易错 / 重点	正确结论
整数除法	`5/2 = 2`（截断），不是 2.5
乘幂中的整数除法	`16**(1/2) = 1`：1/2 先算成 0！
混合运算	先显式类型转换再运算，别依赖编译器自动转换
浮点相等判断	`SQRT(3.0)**2-3.0` 未必等于 0，必须用 `abs(b-a)<1e-6` 之类误差判断
IMPLICIT 规则	I~N 开头默认整型、其余默认实型；每个程序单元都写 `implicit none`
声明位置	声明（含 DATA）必须在所有可执行语句之前
FORMAT 字段不够	输出 ***；整数用 A 格式、类型不符也是常见错误
传址调用	子程序改形参 = 改主程序实参；常量不能传给会被修改的形参
数组越界	下标必须在声明范围内；引用维数必须与声明一致
F90 数组赋值	整体赋值是并行语义：`a(1:9)=a(0:8)` 是平移；DO 循环是串行语义：结果是全 0
按列存储	内层循环对应第一个下标，访问连续内存

易错 / 重点	正确结论
COMMON 对应规则	按位置对应不按名字；只取第 6 个也要声明前 5 个
递归	必须加 recursive；局部变量每次调用独立
write 自动换行	与 C 不同；双引号输出要连写两个双引号
OPEN 的 unit	避开 1、2、5、6（5 键盘、6 屏幕）
经度周期边界	东经 180° = 西经 180°，插值时最后一格与第一格配对
INTENT 属性	只对**形参**有效；只读形参被重新赋值 → 编译错误，OUT 形参未赋值 → 编译警告
省略参数	`OPTIONAL` 声明的形参必须用 `present()` 判断后再使用；按名传参一旦开始写 `名字=`，后面每个参数都要写名字
必须写 INTERFACE	返回数组/指针、按名传参、参数数目不定、ELEMENTAL、超载运算符或赋值号等情形；模块中的过程自带接口
内部函数	`CONTAINS` 后的局部过程只能被包含它的程序单元调用，主程序调用时无需接口
指针未初始化	使用前必须先 `=>` 或 `nullify`，并用 `associated` 检查；编译器不检查
指针数组	不会随过程结束自动释放，必须自己 `deallocate`（动态数组才会自动释放）
链表插入顺序	先让新结点的 next 指向原后继，再改动前结点指针；顺序颠倒会丢失后续数据
NAMELIST	属于声明部分，必须写在执行语句之前且必须取名；变量应先赋初值
优化顺序	先换更好的算法，再谈表达式改写；数组访问要让内层循环走第一个下标
静态 / 动态链接库	静态库把机器码复制进执行文件（用了 module 还要 *.mod）；动态库运行时才调用，便于更新

附录 B　常用内在函数速查

类别	函数
数学	`ABS SQRT EXP LOG LOG10 SIN COS TAN ASIN ACOS ATAN SINH COSH TANH MOD(A,P) MAX MIN`
转换	`INT(A) REAL(A) DBLE(A) CMPLX(X[,Y]) AINT ANINT NINT ICHAR(C) CHAR(I) ACHAR IACHAR`
数组	`ALL ANY COUNT SUM PRODUCT MAXVAL MINVAL MAXLOC MINLOC DOT_PRODUCT MATMUL TRANSPOSE SHAPE SIZE LBOUND UBOUND RESHAPE MERGE PACK UNPACK SPREAD CSHIFT EOSHIFT ALLOCATED`
字符串	`LEN LEN_TRIM TRIM INDEX SCAN VERIFY REPEAT ADJUSTL ADJUSTR`
查询/位	`SELECTED_INT_KIND SELECTED_REAL_KIND KIND HUGE TINY EPSILON PRECISION RANGE PRESENT ASSOCIATED NULL BTEST IAND IOR IEOR IBITS ISHFT`
随机数	`RANDOM_SEED RANDOM_NUMBER`

附录 C　Fortran 90 参考卡精选（fortran90_refcard.pdf）

```
! 过程接口三件套（现代风格）
subroutine foo(a,b,c,d,e,x,y)
  integer, intent(in)  :: a    ! 只读形参
  integer, intent(inout) :: b  ! 读写形参
  integer, intent(out) :: c    ! 只写形参
  real, optional :: d          ! 可选参数，用 present(d) 检查
  character(len=*) :: e        ! 假定长度字符串
  real, dimension(2:,:) :: x   ! 假定形状数组
end subroutine
call foo(1,2,3,e="s",x=a,y=b) ! 关键字实参

! 常用形式
interface operator(+) ... end interface   ! 运算符重载
interface assignment(=) ... end interface
use foo, lname => usename          ! 模块改名引入
use foo2, only: [only-list]       ! 选择性引入
recursive function f(x) ...       ! 递归函数
stop 'message'                    ! 带信息终止
data i,j,k/3*0/                   ! 重复计数赋初值
s2 = s(2:5); s2 = s(:5); s2 = s(5:)   ! 子串
```